

Резюме исполнительного проекта (Executive Summary)

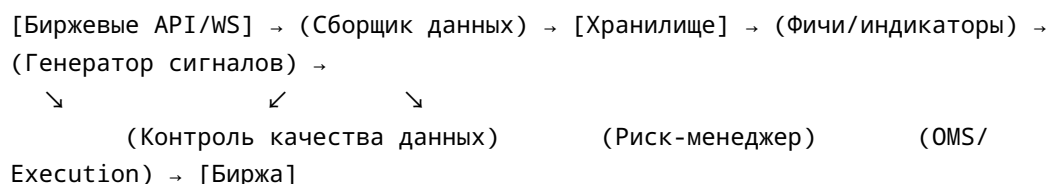
- **Архитектура платформы:** Модульный дизайн, разделяющий сбор данных, расчёт сигналов и исполнение сделок. Реализуется **событийно-ориентированный конвейер** – данные стекаются в реальном времени вместо ночных батчей ¹ ². Используются облачные сервисы (например, Timescale/ClickHouse для хранения) и Python-бэкэнд для анализа.
- **Ключевые риски:** Разрывы или задержки в данных (вебсокеты могут терять события при нагрузке ³), дрейф API (изменения форматов/эндпоинтов), **расхождение состояния позиций** между локальной системой и биржей, искажение модели из-за **drift** (смены рыночного режима) – всё это может привести к убыткам.
- **Ожидаемый профиль ROI:** Целевая стратегия – умеренная доходность с высокой скорректированной на риск эффективностью. При строгом контроле рисков (Sharpe ~1–2) **ожидаемая месячная доходность** может составлять несколько процентов. ROI ограничен отказом от HFT и колокации, но стабильность выше за счёт системного подхода к риску.
- **Необходимые контролы:** Жёсткие **лимиты потерь** (например, автостоп торговли при просадке > X% в день ⁴), ограничение максимального кредитного плеча и размера позиции, *kill-switch* на случай аномалий (например, отключение алгоритма при превышении порога рисков ⁵). Также – проверка баланса перед каждой сделкой, белый список выводимых адресов и аудит логов для прозрачности.

1. Сравнение архитектур (Reference Architectures)

Event-Driven vs Batch: Институциональный подход опирается на **событийную обработку данных**: как только приходят котировки или сделки, они сразу транслируются в систему сигналов. Это снижает задержки и даёт *tick-to-trade* за миллисекунды, необходимое для крипторынков 24/7 ¹ ². *Batch*-подход (например, раз в час/день) проще, но ведёт к упущенным возможностям и задержкам – **«ночные батчи» неприемлемы для realtime криптоаналитики** ². Для единоличного квант-инженера сбалансированным вариантом может быть гибрид: потоковые обновления по WebSocket для критичных данных и периодические батчи для второстепенных расчётов (например, агрегированные метрики раз в минуту).

Монолит vs Микросервисы: Монолитное приложение проще в развёртывании и отладке для одного разработчика – единый кодовый базис (например, один Python-процесс) обрабатывает и данные, и сигналы, и торговлю. Микросервисная архитектура разделяет функции (инжест данных, торговый движок, риски, отчёты) на независимые сервисы, общающиеся через очереди событий. Преимущество микросервисов – **устойчивость и масштабируемость**: падение одного компонента не остановит остальные ⁶. Например, отдельный сервис собирает тики с биржи, другой рассчитывает индикаторы – сбой индикаторного сервиса не мешает сбору данных ⁶. Однако для личной платформы микросервисы усложняют сопровождение (нужен Kubernetes/докеры, мониторинг множества процессов). **Рекомендация:** Начать с *модульного монолита* (разделив логику по модулям внутри одного приложения) и заложить возможность вынесения модулей в сервисы по мере роста нагрузки.

Диаграмма потоков данных: Ниже приведена ASCII-схема типичного пайплайна:



Пояснение: События с бирж (трейды, книги ордеров, свечи) поступают через REST/WebSocket в сервис сбора. Там они проходят очистку/идемпотентную обработку и сохраняются в БД. Далее расчётные процессы извлекают свежие данные, обновляют признаки/индикаторы и генерируют торговые сигналы. Перед выставлением ордеров каждый сигнал проходит проверку рисков (лимиты позиций, VAR и пр.). Ордер-менеджер (OMS) отправляет заявки через API биржи и логирует выполненные сделки. Все компоненты пишут логи и метрики для мониторинга.

Обработка данных: Событийная (on-event) архитектура требует асинхронности: например, поток WebSocket-сообщений обрабатывается немедленно колбэком, обновляя БД и сигналы. *Batch*-архитектура может использовать планировщик (cron/оркестратор) для периодических задач – например, **ежесекундное обновление** свечей и расчёт сигналов по расписанию. Batch-подход проще, но возникает лаг в обнаружении сигналов. В криптовалютах, где движение цен стремительное, **event-driven предпочтительнее**; однако, для тестирования стратегий на дневных данных можно запускать batch-скрипты (Airflow DAG или Prefect flow) раз в сутки, это упрощает отладку.

ASCII-диаграмма данных: В схеме выше показана модульность: сбор данных и торговый execution разделены. Такой подход облегчает отладку (можно тестировать генератор сигналов автономно на исторических данных). Также реализуется **back-pressure**: если хранилище не успевает, очереди/стрим буферизует данные, не давая потерять тики.

Вывод: Для личной платформы предлагается **гибридная архитектура**: один деплой (монолит) с чётко разграниченными подмодулями, работающий в основном в event-driven режиме. Критически важные цепочки (маркет-дата → сигнал → ордер) будут асинхронными и потоковыми. Менее срочные задачи (отчётность, обновление моделей) могут выполняться батчево через встроенный оркестратор. При росте требований можно постепенно выделить микросервисы (например, вынести хранилище TimescaleDB и подключиться к нему удалённо; запустить риск-модуль отдельно для надёжности и т.д.).

2. Слой данных: инжест, хранение, качество (Data Layer)

Сбор рыночных данных: Платформа собирает тиковые данные 24/7 с целевых рынков – спот и бессрочные фьючерсы (swaps). Для этого используются API бирж (REST и WebSocket). **WebSocket предпочтителен для потоковых обновлений** – он обеспечивает push-данные с минимальной задержкой, что критично для частого трейдинга. REST же работает по принципу запрос-ответ и подходит для редких или массовых запросов (например, исторический бэкфилл) ⁷. Идеальный подход – комбинировать: изначально выполнять *backfill* исторических свечей/трейдов через REST, а затем подписаться на WebSocket для получения новых данных по мере их появления. В 2025 году **боты реального времени используют WebSocket для цен и стаканов, а боты с дневными сигналами могут работать по REST** ⁸. Практически, можно применять агрегатор API **CCXT** (поддерживает ~100 бирж) для унификации REST-запросов. Для WebSocket CCXT имеет

ограниченную поддержку, поэтому возможно использование native-библиотек бирж (OKX v5 SDK, Binance API) или асинхронных обёрток (например, **АИОНТПР** + CCXT Pro).

Идемпотентность и метки (watermark): Ключевая задача – не пропустить и не задвоить данные. При запуске сборщик выполняет историческую загрузку (например, всех свечей за год) и записывает их в БД. Далее он **отмечает последнюю полученную метку времени** и переходит в режим real-time. Новые данные запрашиваются либо с указанием `since=last_timestamp` (для REST) либо приходят по стриму. Система должна игнорировать дублирующиеся записи и соблюдать порядок. Лучшая практика – использовать *идемпотентные upsert*-операции в БД (ON CONFLICT DO UPDATE), либо сохранять полученные уникальные ID (например, `trade_id`) последних обработанных записей. Так достигается **exactly-once** загрузка: дубли отфильтровываются, а пропущенные интервалы дозапрашиваются автоматическим бэкфиллом. Например, open-source сборщик данных JohnBdfilio выполняет: (1) *исторический бэкфилл с указанной даты*, (2) *переход к сбору новых свечей строго по границам интервала*) ⁹. При этом **удаляются дубликаты записей** – файл данных дополняется новыми строками, а повторяющиеся отсекаются ¹⁰. В реализации можно хранить для каждой биржи/тикера *watermark* (последний UTC-времestамп или номер сделки) и ретраить до успеха: если связь разорвалась, повторить запрос с последней успешной точки.

Стратегия на случай сбоя соединения: WebSocket-соединения ненадёжны: при всплеске волатильности могут быть дропы или задержки ³. Поэтому сборщик должен иметь *reconnect-логику*: при обрыве сразу переподключиться и запросить пропущенные данные через REST. Например, если стрим свечей пропал на 5 минут – сделать REST-запрос этих 5 минут, чтобы не было **data gap**. Каждая свеча/сделка снабжается *биржевым временем*, которое позволяет детектировать пропуски (если новая свеча пришла с большим разрывом во времени – нужно дозапросить недостающие интервалы).

Хранилище (БД) для маркет-данных: Рассматривается несколько опций: - **TimescaleDB (PostgreSQL + расширение)** – реляционная СУБД, оптимизированная под временные ряды. Поддерживает *гипертаблицы* (автоматическое секционирование по времени), сжатие старых партиций, continuous aggregates. Преимущества: знакомый SQL-интерфейс, транзакционность (ACID), удобные *time_bucket* функции и *JOIN* с метаданными ¹¹ ¹². Масштабируемость – вертикально (Postgres можно шардировать через Citus, но Timescale в open-source в основном на один узел). - **ClickHouse** – колоночная СУБД для аналитики. Очень высокое быстродействие агрегаций и вставки батчами (сотни тысяч строк/сек). Поддерживает *distributed tables* и шардинг для горизонтального масштабирования ¹³. Однако, не имеет полной транзакционности (insert-only, eventual consistency), что может привести к дубликатам или расхождениям при сбоях ¹⁴. Отлично подходит для офлайн-аналитики больших объёмов (терабайты тиков), но управление сложнее (свой движок, нет полноценного SQL JOIN как в Postgres). - **Parquet + DuckDB** – файловое хранение. Идея: хранить данные по дням в файлах Parquet (колоночный формат), а для запросов использовать **DuckDB** – встроенную БД, умеющую читать множество Parquet как одну таблицу. Такой подход очень экономичен (файлы сжаты, нет отдельного сервера БД) и **быстр в аналитических запросах** на локальных данных. Проект JohnBdfilio как раз сохраняет OHLCV по дням в Parquet-файлы (структура папок: `exchange/symbol/interval/date.parquet`) ¹⁵. Минусы: сложно обеспечить непрерывную запись в Parquet в real-time (скорее это батчевый сценарий), нет сетевого доступа (DuckDB – процесс в памяти). Поэтому Parquet/ DuckDB хорошо применять для *бек-тестов* и офлайн-аналитики, а для 24/7 работы лучше Timescale или ClickHouse. - **Vanilla PostgreSQL** – возможен как базовый вариант (Timescale – это надстройка над PG). Без Timescale, Postgres тоже может хранить миллионы строк, но потребуются ручное партиционирование таблиц (например, по месяцу) для производительности. В высокочастотном

сценарии PG уступает специализированным TSDB: на порядки меньше скорость вставки и агрегации, если не оптимизировать.

Производительность и масштабы: Бенчмарки показывают, что **ClickHouse быстрее Timescale** на агрегатных запросах по большим денормализованным данным (например, ClickBench test), однако на типичных *финансовых* задачах с фильтрацией и JOIN Timescale может превосходить ¹⁶. В тесте TigerData RTABench, имитирующем реальные аналитические запросы, TimescaleDB показал скорость в 1.9x выше ClickHouse (на нормализованных данных с выборочной выборкой), хотя на синтетическом ClickBench был медленнее ¹⁶. Вывод: Timescale лучше для онлайн-аналитики и сложных запросов (преимущество PostgreSQL optimizer), ClickHouse – для тяжёлых исторических вычислений по колонкам. DuckDB близок к ClickHouse по скорости на одном узле, но рассчитан на локальную работу (отсутствие сетевого сервера).

Партиционирование и TTL данных: Время серии данных растёт бесконечно, поэтому нужно планировать очистку или архивацию. TimescaleDB позволяет задавать *политику ретеншена* – автоматическое удаление или сжатие партиций старше N дней. Например, можно хранить сырые тики 1 год, далее удалять или агрегировать их в 1H свечи. ClickHouse поддерживает TTL для партиций – можно при создании таблицы указать столбец времени и срок хранения, после которого строки будут удалены. Также можно периодически переводить старые данные в Parquet-архивы на объектном хранилище. Для DuckDB ретеншен – вручную удалять старые файлы.

Схемы данных: Рекомендуется проектировать явные схемы таблиц под основные сущности: - Таблица свечей (OHLCV) с индексом по времени; - Таблица сделок (tick trades) с ключом (exchange, symbol, trade_id) – для уникальности; - Таблица заявок/позиций – для журнала и текущего состояния. Ниже приведены минимальные схемы (SQL DDL):

```
-- Свечи OHLCV
CREATE TABLE ohlcv (
  exchange TEXT,
  symbol TEXT,
  interval TEXT,      -- напр. '1m', '1h'
  ts          TIMESTAMPTZ, -- начало интервала
  open        NUMERIC(18,8),
  high        NUMERIC(18,8),
  low         NUMERIC(18,8),
  close       NUMERIC(18,8),
  volume      NUMERIC(18,8),
  PRIMARY KEY(exchange, symbol, interval, ts)
);
```

```
-- Сырые сделки (trade ticks)
CREATE TABLE trades (
  exchange TEXT,
  symbol TEXT,
  trade_id BIGINT,
  ts          TIMESTAMPTZ,
  price       NUMERIC(18,8),
  size        NUMERIC(18,8),
  side        TEXT,      -- 'buy'/'sell'
```

```
PRIMARY KEY(exchange, symbol, trade_id)
);
```

```
-- Позиции и PnL
CREATE TABLE positions (
    symbol      TEXT PRIMARY KEY,
    amount      NUMERIC(18,8), -- текущий размер позиции (+ long, - short)
    avg_entry    NUMERIC(18,8), -- средняя цена входа
    unrealized_pl NUMERIC(18,8), -- текущий плавающий PnL
    realized_pl  NUMERIC(18,8) -- зафиксированный PnL
);
```

(Примечание: TimescaleDB использует `ts TIMESTAMPTZ` как *time-partition key*, ClickHouse – можно определить *PARTITION BY toYYYYMM(ts)*. Для больших таблиц стоит создавать вторичные индексы, например, на *(symbol, ts)* если часто выборка по инструменту.)

Контроль качества данных: Система должна **гарантировать целостность данных**: все поля соответствуют контракту схемы, нет пропущенных значений. В конвейере инжеста можно встроить валидаторы: проверять, что цена и объём не отрицательные, время монотонно не убывает, дубликаты отбрасываются. При нарушении – логировать и алертировать. Важен и **time synchronization**: сервер, на котором работает бот, должен иметь точное время (NTP-синхронизация). Даже для несинхронных стратегий это критично: рассинхронизация часов может привести к неправильным меткам в БД и проблемам с аутентификацией API (некоторые биржи требуют, чтобы локальное время запроса не отличалось более чем на X секунд). Поэтому рекомендуется настроить демоны NTP (например, Meinberg NTP на Windows или chrony на Linux) ¹⁷. Системный час должен проверяться ежедневно.

Логи lineage: Для любых изменений данных (например, корректировка свечи из-за биржевого пересчёта) должна сохраняться *происхождение*. Например, если биржа отозвала ошибочную свечу или произошёл rollback блока (в случае on-chain данных) – система вносит изменение и пишет в журнал: *"Candle for BTC-USD 2025-10-20 12:00 adjusted due to exchange correction"*. Такие логи полезны для аудита и отладки стратегий.

В целом, слой данных опирается на принципы **идемпотентности и полноты**: любые запросы могут повторяться без вреда (например, повторный импорт дня не создаст дубликатов), а при сбоях данные догружаются до требуемой точки. Хорошая практика – периодический *снэпшот состояния*: раз в день сверять агрегаты (например, объём торгов за сутки) с эталонными данными биржи, чтобы убедиться в отсутствии рассинхронов.

3. Расчёт признаков и генерация сигналов (Feature & Signal Engine)

Технические индикаторы: Платформа вычисляет стандартные индикаторы TA и специализированные признаки на основе собранных данных. Для этого целесообразно использовать проверенные библиотеки: **pandas** и **NumPy** для основных агрегатов, набор индикаторов из **TA-Lib** (через обёртки вроде *pandas-ta*) или **VectorBT**. PandasTA предоставляет 100+ индикаторов (MA, RSI, MACD и т.д.) оптимизированно, а VectorBT позволяет вычислять

индикаторы по массивам очень быстро (с использованием Numba) ¹⁸. Кастомные индикаторы (например, проприетарные сигналы) могут быть реализованы как функции pandas rolling.

Режимы рынка и волатильность: Помимо классических индикаторов, важны **фичи режима рынка**: напр., волатильность за период, трендовость vs флэт, метрики ликвидности. Исследования показывают, что криптовалюты демонстрируют *кластеризацию волатильности* аналогично акциям ¹⁹ – периоды высокой волатильности сменяют друг друга. Поэтому система может включать детектор режимов: напр., классификация волатильности (низкая, средняя, высокая) с помощью скользящего окна и порогов или HMM-модель. **Cointegration и парный трейдинг:** Если в стратегии используются несколько активов, можно вычислять их коинтеграцию. Последние исследования подтверждают, что стратегии на основе коинтеграции (парный трейдинг) **эффективны на L1-токенах, особенно в периоды волатильности** ²⁰ – платформе стоит иметь модуль проверки коинтеграции (например, тесты Энгла-Гранджера) и расчёта спреда.

Формирование целевых меток (labeling): Для моделирования (например, обучения ML) нужно присваивать событиям целевые метки (прибыльный сигнал/убыточный и т.п.). Здесь пригодятся методы из *Advances in Financial ML*: **тройной барьер (Triple Barrier)** для классификации исхода сделки по достижению определённых профит/стоп границ, методы meta-labeling. Это защищает от смещения выборки и учитывает время удержания позиции ²¹. Также можно помечать данные **режимами** (bull/bear) по каким-то критериям, чтобы модель могла учитывать контекст.

Проверка утечек данных: *Model risk* прежде всего связан с **утечкой информации (data leakage)**. Нужно гарантировать, что ни при расчёте признаков, ни при бэкteste модель не заглядывает в будущее. Например, индикатор скользящего среднего на момент t должен быть рассчитан только по данным $< t$. Любые функции, требующие будущих значений, недопустимы. Утечка делает результаты нереалистично хорошими ²². Будут внедрены тесты на утечки: скользящее окно для всех агрегатов, смещение признаков на 1 шаг назад при тренировке модели (prevent lookahead). В коде бэктеста это значит: никакого .iloc+1 для цены закрытия будущей свечи и т.п. – такие вещи проверяются специально. Также используется **purging/ embargo** при разделении выборок во времени ²³ ²⁴ – после периода обучения часть данных выкидывается (embargo), чтобы удалённые по времени метки не влияли на тест.

Версионность признаков и моделей: По мере эволюции платформы индикаторы и стратегии будут меняться. Важно внедрить *versioning* – например, хранить в метаданных модели, с какими версиями признаков она обучена. Инженер может использовать **MLflow или DVC** для отслеживания экспериментов: сохранять, что Модель_1 обучена на признаках v1.2. Если затем логика расчёта RSI изменилась, это будет новая версия признаков (v1.3), и старую модель нужно переобучить. Это предотвращает скрытые несоответствия между live-сигналами и моделью.

Rolling-обучение и онлайн-обновление: Поскольку рынки меняются, система должна поддерживать **скользящее обучение**. Например, модель обучается на окне последних N месяцев и переобучается ежеквартально (walk-forward approach). *Walk-forward анализ* – стандарт против overfitting: берётся начальный период для обучения, затем последующий для теста, потом окно сдвигается ²⁵. Этот механизм (или расширяющееся окно) встроен в план исследований. Для алгоритмических стратегий без ML – параметры калибруются похожим образом (например, оптимизация на 2024, проверка на Q1 2025, потом пересмотр параметров).

Вывод сигналов: *Engine* генерирует торговые сигналы на основе правил стратегии или модели. Например, простая стратегия: сигнал BUY, если $MA_{fast} > MA_{slow}$, и SELL, если наоборот. Сигналы могут быть дискретными (-1, 0, +1) или в виде скоров вероятности. Важно, что генерация

сигналов отделена от исполнения – engine может выдавать сигнал, даже если сейчас его нельзя исполнить (это проверит блок риска/EMS). Сами сигналы хранятся в базе (в таблице signals с меткой времени и содержимым), что помогает отладке и анализу: всегда можно посмотреть, какие сигналы были, даже если они не исполнены.

Пример псевдокода расчёта сигналов:

```
# Получение последних данных
df = fetch_ohlcv(symbol="BTC/USDT", interval="1m", limit=500)
# Расчёт индикаторов
fast_ma = ta.SMA(df['close'], timeperiod=50)
slow_ma = ta.SMA(df['close'], timeperiod=200)
# Сигнал пересечения MA
signal = 0
if fast_ma[-1] > slow_ma[-1]:
    signal = 1 # покупка
elif fast_ma[-1] < slow_ma[-1]:
    signal = -1 # продажа
# Сохранение сигнала
store_signal(symbol="BTCUSDT", time=df.index[-1], signal=signal)
```

Этот модуль можно усложнять, включая проверку режима (напр., торговать только в трендовые периоды) и другие фильтры. Для ML-сигналов pipeline будет иным: сначала формирование *feature matrix*, затем инференс модели (например, `model.predict_proba(features)`), и затем принятие решения на основе прогноза и порогов.

Unit-тестирование индикаторов: Для предотвращения ошибок любая новая фича сопровождается тестом: на известном мини-наборе цен рассчитываем индикатор и сравниваем с эталонным значением (например, сравнить наш SMA с результатом `pandas_ta`). Также делается *leakage test*: убедиться, что индикатор на *t* не зависит от цены на *t* (например, некоторые библиотеки могут по умолчанию использовать будущее значение – этого нельзя допустить).

Итого, движок признаков/сигналов обеспечивает **богатый набор показателей рынка** (TA-индикаторы, статистические фичи) и выпуск сигналов, избегая при этом ловушек оверфиттинга и лукахаед-байаса ²⁶. Все сигналы помечаются временем генерации и используемой моделью/стратегией для возможности пост-анализа результатов.

4. Разработка стратегии и валидация (Strategy Research & Validation)

Бэктестинг: Критический этап – проверка стратегий на исторических данных. Платформа будет иметь модуль бэктестинга, способный **симулировать сделки по историческим котировкам**. Необходимо учитывать реалистичные условия: задержки, спреда, проскальзывание, комиссии, финансирование. Существуют готовые фреймворки: - **VectorBT** – современный бэктестер, позволяющий как *векторизованный* (на массивах) просчёт, так и *событийный* с колбэками ¹⁸. Отличается высокой скоростью: до 1еб ордеров за 0.1 секунды на Apple M1 ¹⁸. Поддерживает указание комиссий и вывод подробной статистики. - **Backtrader** – популярный Python-фреймворк с объектно-ориентированным API, более простой для начинающих. Поддерживает мульти-

инструментные стратегии, имеет встроенные индикаторы и возможно подключение к живой торговле (IB, Oanda) ²⁷ ²⁸. Минус – медленный на больших объёмах (нет векторизации, чистый Python) ²⁹. Для небольших данных приемлемо, но на тиковых историях может «тормозить». - **Lean (QuantConnect)** – институциональный движок бэктестинга (open-source на C#). Очень мощный и проверенный (поддерживает акции, опционы, фьючерсы, форекс), с продвинутыми моделями исполнения. Однако, для крипто нужна интеграция с данными, и запуск Lean сложнее (лучше через облако QuantConnect).

Для нашей цели оптимально использовать **VectorBT** или собственный модуль на его основе. VectorBT позволяет задать свои правила исполнения: например, модель частичного заполнения ордера и проскальзывания. Можно реализовать *event-driven backtest*: прокручивать исторический поток (например, тиков) и в каждый момент принимать решения/менять состояние. Это повторяет логику реальной работы и даёт наиболее точную симуляцию.

Моделирование исполнения и издержек: Нужно закладывать **комиссии биржи** – фиксированные (spot) или плавающие (futures maker/taker). В бэктесте комиссия учитывается при каждой сделке (уменьшая баланс). Например, VectorBT при создании портфеля может принимать параметр `fees=0.001` (0.1%) ³⁰ ³¹. Проскальзывание (slippage) – разница между теоретической ценой сигнала и реальной ценой исполнения. На высоколиквидных парах можно моделировать slippage как случайный $\pm X$ bps от цены. Более сложный подход – использовать реальные спреды из ордербука: например, для каждого сделанного в бэктесте ордера смотреть ордербук в тот момент и «исполнять» по соответствующей цене (симуляция *market impact*). Если нет таких подробных данных, можно задавать консервативную надбавку (например, покупка по цене на 0.05% выше close). **Финансирование (funding rates)** на бессрочных фьючерсах: раз в 8 часов позиции платят/получают финанс. Бэктестер должен учитывать это: моделировать раз в 8h дебет или кредит на PnL исходя из текущей позиции * ставки. Например, QuantConnect Lean поддерживает забор funding rate через отдельный data-feed и корректирует позицию ³² ³³. В нашем случае можно реализовать проще: при пролистывании исторических данных, каждые 8 часов уменьшать баланс на сумму `PositionSize * FundingRate * Interval`.

Anti-overfitting при тестировании: Большая опасность – подобрать стратегию под историю (overfit) и получить плохой результат на новых данных. Поэтому используется **прогрессивное тестирование**: - *Walk-forward анализ*: период N (например 1 год) для оптимизации параметров → следующий период M (напр. 3 месяца) для out-of-sample проверки. Цикл повторяется по всей истории. Это позволяет увидеть устойчивость параметров. - **Комбинаторное кросс-валидация (CPCV)**: метод Лопеса де Прадо ³⁴, генерирующий множество разбиений train/test с *purging* перекрывающихся данных. Он более надёжно оценивает стратегию при коротких рядах. Реализовать CPCV можно скриптом, но ресурсозатратно. Для ключевых стратегий стоит его применить – так выявятся случаи, когда стратегия случайно заработала на одном отрезке, но проваливается на большинстве других.

- **Nested CV**: если стратегия включает обучение моделей или оптимизацию параметров, то необходимо двойное разбиение: внешнее для оценки и внутреннее для подбора параметров. Например, при подборе окна SMA мы внутри train-сегмента делаем еще K-fold CV для выбора лучшего окна, а потом проверяем на test-сегмент. Это предотвращает утечку информации из теста при подборе параметров.

Атрибуция результатов: После прогонов стратегии важно понять, **за счёт чего получен PnL**. Поэтому отчёты бэктестера включают: - *Factor attribution*: если стратегия сочетает несколько факторов (например, импульс + carry), нужно оценить вклад каждого. Это делается либо *без одного фактора* тестами, либо регрессией PnL на факторовые доходности.

- *Event tagging*: пометка каждой сделки особыми событиями (например, «шпилька

волатильности», «день объявления ставки ФРС»). Затем анализируется, какие события приводили к наибольшим убыткам или прибыли. Например, можно обнаружить, что бот стабильно теряет в дни экспирации фьючерсов – тогда добавить условие отключаться в эти дни.

- *Decomposition of drawdowns*: разбирается крупнейшая просадка – какие пары/сегменты времени дали основной вклад. Это помогает выявить слабые места стратегии (например, стратегия хорошо работает в тренде, но большой даунстрим случился на флэте – значит, нужен фильтр).

Показатели и метрики: Валидация включает стандартные метрики: CAGR, Sharpe, Sortino, max Drawdown, Calmar. Для риск-стратегий – *Stability* (насколько равномерно стратегия зарабатывает), коэффициенты инфо-отношения относительно бенчмарка (BTC). Желательно реализовать *релизные критерии*: стратегия считается приемлемой для запуска, если на истории Sharpe > 1, maxDD < 20%, продержалась >= 3 лет без критического спада и прошла все out-of-sample проверки. Такая формализация защищает от запуска сырой переоптимизированной идеи.

Инструментарий: Можно использовать **те же фреймворки** (VectorBT, PyFolio, empirical) для генерации отчётов. VectorBT, например, имеет встроенные методы для статистики стратегий и рендеринга графиков equity. Это ускоряет исследование: код стратегии раз – а тестировать десятки вариаций можно быстро, пользуясь speed VectorBT (Numba) вместо медленных Python-циклов ³⁵.

Протокол исследований: Каждая идея (стратегия) должна быть задокументирована: допущения, гипотезы, результаты тестов. Желательно вести *ресерч-ноутбуки* (Jupyter с прозаическим описанием и кодом). Это создаст базу знаний: спустя время инженер сможет пересмотреть, почему был отвергнут подход X или подтверждён подход Y.

Итого, фаза разработки и валидации стратегии – это итеративный процесс с максимальным упором на предотвращение переобучения. Использование строгих техник CV и атрибуции, описанных выше, повышает вероятность, что на продакшене стратегия сохранит показанные на истории характеристики ²⁶ ³⁶.

5. Риск-менеджмент, размер позиции и лимиты (Risk, Position Sizing, Limits)

Методы расчёта размера позиции: В платформе должны быть реализованы несколько вариантов алгоритмического управления капиталом: - *Фиксированная доля капитала (fixed-fraction)*: классическое правило – рискнуть на каждой сделке не больше N% депозита (обычно 1-2% ³⁷). Например, если счёт \$10k и риск-процент 2%, максимальный убыток на трейд – \$200. Исходя из этого и стоп-лосса рассчитывается объём позиции. - *Критерий Келли*: для стратегий с оценкой вероятностей можно применять Kelly для оптимального роста. Но полный Kelly слишком агрессивен, обычно берут долю (0.5 Kelly) чтобы снизить волатильность капитала ³⁸. Платформа может вычислять коэффициент Келли на основе исторической винрейта и payoff, и устанавливать размер позиции = $f \text{баланс}$, где f – доля от Kelly (например 0.5). - Risk Parity (волатильностное выравнивание): если одновременно торгуются несколько активов, позиции масштабируются так, чтобы каждый внес одинаковый вклад в риск портфеля. Пример: целевая волатильность портфеля 10%, BTC имеет 80% годовую волатильность, ETH – 100%; тогда доля капитала на BTC ~12.5%, на ETH ~10% для равного вклада. При запуске одной стратегии на разные рынки можно еженедельно пересчитывать веса согласно обратной волатильности. - CVaR-targeting: продвинутый подход – масштаб позиций так, чтобы ожидаемый короткий fall (CVaR) не превышал заданного. Например, Целевой дневной CVaR 1% капитала: система моделирует текущий

риск (через историческое/монте-карло распределение PnL) и уменьшает позиции если ожидаемый $\text{expected shortfall}^* > 1\%$. Этот метод сложнее, но эффективен в учёте хвостовых рисков.

Пользователь может настроить метод – например, для простой стратегии выбрать фиксированный процент риска на трейд, а для портфельной – risk parity.

Ограничения и контроль риска: Платформа внедряет **многоуровневые ограничения**: - *Ограничение плеча:* суммарное используемое плечо (маржинальное) не более X. Например, если $X=3$, а на фьючерсах плечо 10x, бот не будет использовать весь доступный левередж (это сохраняет запас на волатильность). Ограничение может быть и на каждую позицию (не более 2x на альткойны, 5x на BTC) и на портфель. - *Максимальный размер позиции по инструменту:* напр., не более 20% дневного объёма рынка, чтобы не влиять на цену (для крупных капиталов). В нашем случае, одиночный инженер вряд ли превысит это, но механизм на месте. - *Stop-loss и дневные лимиты убытков:* каждый ордер может иметь стоп-лосс (который платформа выставляет сразу после входа в позицию). Кроме того, прописывается *daily stop*: если суммарный убыток за сутки превышает Y%, все позиции закрываются и торговля прекращается до следующего дня ⁴. Это аналог *circuit breaker*: предотвратить эмоциональную/алгоритмическую спираль потерь. Пример: установить дневной стоп 5%. Тогда при просадке -5% бот автоматом отключается на 24h. Многие проп-фирмы требуют этого ³⁹. - *Ограничение по экспозиции:* устанавливается суммарная величина позиции по коррелированным активам. Например, если торгуются BTC и ETH, можно лимитировать что суммарная позиция (приведённая к USD) не превышает \$N. И/или лимит по секторам (DeFi токены вместе не больше X% портфеля). - *Circuit breakers рыночных условий:* если рынок аномальный – например, волатильность VIX криптоиндекса > некоторого порога или ликвидность упала – бот может приостановить торговлю. Аналогично, при сбоях бирж (если API не отвечает или задержки > X) включается *fail-safe*: закрыть активные позиции (если возможно) и ждать восстановления.

Pre-trade проверки: Перед отправкой любого ордера система проходит чек-лист: 1) **достаточность свободного баланса** для маржинальных требований, 2) ордер не нарушает лимиты (вышеописанные, например, не открывает новую позицию, если уже достигнут дневной лимит потерь), 3) проверка связи (например, если связь с биржей лагает >5с, лучше не отправлять ордер, чтобы не получить старую цену). Только при выполнении всех условий ордер идёт в биржу. Если что-то не так, сигнал игнорируется или ставится в очередь до улучшения условий.

Kill-switch и аварийное отключение: Платформа имеет глобальную кнопку (и автоматический триггер), чтобы **срочно остановить торги**. Например, обнаружен баг или подозрение на компрометацию – через kill-switch снимаются все активные ордера и блокируется отправка новых. Интересно, что на уровне бирж тоже появились kill-switch для участников – например, NASDAQ внедрила в 2014 году kill-switch на уровне биржи для отсечения всех ордеров при превышении заданного порога риска ⁵. У нас – свой: если бот уходит в неадекватную зону (например, 10 подряд странных сигналов за секунду), или по команде оператора – мгновенно выключаем торговый модуль.

Реконсильция позиций: Не реже, чем раз в N часов, система сверяет своё представление позиций/баланса с биржевым. Это важно: локально бот ведёт учёт выполненных ордеров, но возможны рассинхроны (например, ордер частично исполнился именно в момент перебоя связи – локально это могли не учесть). Поэтому реализуется **фоновая задача reconcile**: запрашивает через API текущее состояние счёта (активные ордера, позиции) и сравнивает с внутренним. Если обнаружены расхождения, они логируются и приводятся к факту биржи (например, бот считал позицию = 0, а на бирже висит 0.1 BTC – значит, берем биржевое значение и вычисляем, когда это

произошло, корректируем историю). Без этого контроль риска может дать сбой (думал, позиций нет и открыл новые, а они были).

Журналирование риска: Каждый раз, когда стратегия хочет открыть позицию, создаётся запись: что попросили, прошло ли риск-правила, какой апрув. Это audit-trail: если вдруг бот не совершил сделку, можно посмотреть – возможно, отфильтровал риск-модуль.

В итоге, уровень риска интегрируется по всей цепочке: **сигнал → риск-чек → исполнение**. Это предотвращает импульсивные/ошибочные действия алгоритма и защищает капитал от экстремальных ситуаций. Консервативное управление позицией (не более 1-3% риска на сделку ³⁷) и строгие стоп-механизмы – must-have для институционального подхода, даже в личной разработке.

6. Ордер-менеджмент и исполнение (OMS/EMS & Execution)

Типы ордеров и время жизни: Платформа должна уметь выставлять основные типы ордеров: - *Лимитный (Limit)*: с определённой ценой, может стоять в книге до исполнения или отмены. Поддерживаются доп.флаги: **Post-Only** (разместить только как мейкер). Post-Only гарантирует, что ордер не снимет ликвидность – если при размещении он может сразу исполниться как тейкер, биржа либо отменит, либо изменит цену на один тик ⁴⁰. Это важно для избегания тейкерской комиссии и проскальзывания. - *Маркет*: исполняется по текущей лучшей цене немедленно (может проскользнуть). Обычно для бота лучше избегать, кроме случаев стоп-лоссов. - *Immediate-Or-Cancel (IOC)*: частично похож на маркет – исполняется немедленно по заданной (или лучше) цене, **разрешая частичное исполнение**; любую неисполненную часть сразу отменяет ⁴¹. Используется, когда нужно быстро взять ликвидность, но не хочется, чтобы остаток торчал в стакане. - *Fill-Or-Kill (FOK)*: исполнить полностью сразу или отменить. Частичные исполнения не допускаются ⁴². Полезно, если нужна точная размером сделка или на неликвидном рынке (иначе частичная сделка может быть бесполезна). - *Stop-Limit / Stop-Market*: для автоматической отправки заявки при достижении стоп-уровня. Например, стоп-лосс или тейк-профит. Бот может сам обрабатывать условия или использовать биржевые стоп-ордера, если биржа поддерживает.

В реал-тайм торговле важно уметь работать с **частичными исполнениями (partial fills)**: например, отправили лимит на 10 BTC, исполнилось сразу 3 BTC, остальное висит. OMS должен зафиксировать частичное исполнение и уменьшить остаток ордера. Многие биржи сообщают об этом через WebSocket-события или REST в ответе. Система хранит в ордер-журнале состояние: New → Partially Filled (кол-во) → Filled или Canceled. Логика управления ордерами должна учитывать, что, например, может прийти событие *cancel* с *filled* = 3 BTC, означающее, что 7 BTC отменено, 3 исполнилось.

Алгоритмы исполнения (execution algos): Для крупных ордеров или для минимизации импакта применяются простейшие алгоритмы: - *TWAP (Time-Weighted Avg Price)*: разбивает большой ордер на множество мелких и исполняет равномерно за заданное время ⁴³. Например, надо продать 100 ETH за час – бот каждые 5 минут продаёт ~8.33 ETH по рыночной цене. Это сглаживает влияние на рынок. - *VWAP*: учитывает объёмы – исполняет пропорционально ожидаемому распределению объёма торгов в течение интервала. Например, больше продаёт в периоды высокого общего объёма, меньше – в тихие минуты, пытаясь получить цену ближе к Volume Weighted Average. Требуется оценки профиля объёмов. - *Iceberg*: отправляет лимитные ордера по частям, скрывая полный объём ⁴⁴ ⁴⁵. Биржа как таковая может поддерживать iceberg, либо бот сам выставляет новую лимитку после исполнения предыдущей. - *Sniping/Maker algorithms*:

отправляет post-only лимиты чуть лучше текущих цены, чтобы заработать rebate и быстро закрываться.

Наша платформа не ориентирована на HFT, но **TWAP можно реализовать для медленного входа/выхода** при больших суммах (или при ребалансировке портфеля). Например, если стратегия дала сигнал купить на \$100k, лучше не покупать всё мгновенно, а распределить на десятки заявок за несколько минут.

Управление задержками и ошибками: Компонент исполнения должен работать асинхронно и очень надёжно: - Все запросы на ордера идут с таймаутами. Если API не ответил за, скажем, 3 секунды – считать, что заявка неизвестно выставлена или нет, и перейти в режим reconcile (через пару секунд проверить через REST активные ордера). - Повторная посылка заявки: чтобы избежать дублей (например, непонятно, прошёл первый запрос или нет), используется **идемпотентный ключ** – большинство бирж позволяют задавать `client_order_id`. Если мы повторно отправим тот же `client_id`, биржа либо скажет «ордер с таким id уже существует» либо примет, но второй с тем же id не создаст. Например, Binance API поддерживает параметр `newClientOrderId` – мы можем генерировать UUID для каждой заявки и тем самым отличать их. - **Ретрай по градации (retry ladder):** если лимитный ордер не исполняется длительное время, можно постепенно улучшать цену. Например, хотим купить, лимит стоит по 100.0 и висит 30 секунд неисполненным – бот может снять и переставить по 100.1 (на шаг ближе к рынку), и так далее. Такой «лестничный» ретрай повышает шанс исполнения, контролируя проскальзывание. Аналогично при продаже – если не исполняется, немного снизить цену. Параметры (шаг цены, время ожидания) настраиваются. - **Журнал ордеров:** OMS ведёт полную историю: для каждого `client_order_id` – время отправки, параметры, ответ (успех/ошибка), время и объём исполнения, время отмены и причину. Это необходимо для аудита и отладки. В случае любой несоответствия с биржей, по журналу можно воспроизвести события.

Реалтайм-обновление статуса: Подсистема должна оперативно получать апдейты об исполнении. Обычно это WebSocket stream (например, Binance User Data Stream) или REST long-poll. Это нужно, чтобы сразу реагировать – например, после **полного исполнения заявки** выставить связанные ордера (стоп, тейк). Если WS недоступен, можно периодически (раз в секунду) дёргать REST `openOrders` / `myTrades`.

Частичный недозакрытие (Reconciliation): В случае, когда по какой-то причине автоматизм нарушен (например, бот думал, что закрыл позицию, а часть осталась), должна быть процедура: выявить неучтённую позицию и закрыть её по рынку либо сообщить оператору. Это часть риск-менеджмента/OMS: не должно зависнуть «осиротевших» позиций.

Пример псевдокода исполнения заявки и последующей сверки:

```
def place_order(side, price, size, type="LIMIT"):
    cid = generate_uuid() # уникальный client id
    try:
        resp = api.place_order(side=side, price=price, size=size,
                                client_id=cid, type=type)
        order_id = resp["orderId"]
        status = resp["status"]
        log_order(order_id, cid, side, price, size, status)
    except TimeoutError:
        # неизвестно, выставился или нет, пометим для проверки
```

```

log_error("Order placement timeout", cid)
schedule_reconcile_order(cid)

def reconcile_orders():
    open_orders = api.get_open_orders()
    for tracked in local_active_orders:
        if tracked.cid not in open_orders:
            # ордера нет на бирже, значит исполнился или отменён
            trade = api.get_order(tracked.order_id)
            update_local_order(tracked, trade.status, trade.filled)

```

Здесь `schedule_reconcile_order` – планирует через короткий интервал проверить судьбу ордера (в случае таймаута).

Поддержка нескольких бирж: OMS должен быть абстрактным – возможно, подключены сразу OKX, Binance, Coinbase. Через CCXT это унифицировано: достаточно задать exchange id. Но детали (тип ордеров, параметры) могут отличаться – для этого делается уровень адаптера. Можно, например, написать `ExchangeInterface` с методами `place_order()`, `cancel_order()`, реализовать для каждой биржи. Тогда стратегия не зависит от конкретной биржи – она вызывает интерфейс, а под капотом будет CCXT с правильными параметрами (например, Binance требует price как строку, а Coinbase – в другом формате).

Учёт биржевых ограничений: Перед отправкой ордера OMS проверяет: размер не меньше минимального лота биржи (у каждой пары есть minSize), цена кратна шагу (tick size). Эти параметры можно получить из API биржи или захардкодить. Например, нельзя отправить 0.0005 BTC на Binance, если minLot 0.001. Бот автоматически округлит до разрешённого минимума.

Наконец, исполнение – самое чувствительное место, оно работает под нагрузкой. Нужно много логирования (в том числе задержки ответа API), и желательно метрики: процент ордеров, исполненных как мейкер vs тейкер, среднее проскальзывание. Эти данные потом идут в анализ качества исполнения.

7. Автоматизация процессов и наблюдаемость (Automation & Observability)

Оркестрация задач: Даже в персональной платформе много регулярных задач – сбор данных, перерасчёт фич, обучение моделей, отчёты. Вместо разрозненных cron-скриптов лучше использовать **оркестратор** (Airflow, Prefect, Dagster). Он позволит декларативно описать зависимости и графы задач. Например: - DAG `DataIngest`: таски по биржам/тикерам, которые собирают новые данные (каждый минутный запуск инкрементально) – можно реализовать в Airflow как Sensors + Tasks или в Prefect как flow с `.map` по списку пар. - DAG `Features`: раз в день пересчитать долгосрочные признаки (например, волатильность 30д для всех инструментов). - DAG `Training`: раз в неделю обучить модели (если используются ML). - DAG `LiveTrade`: процесс, который запускает торговый бот (может быть не DAG, а persistent service – тогда оркестратор лишь контролирует запуск/перезапуск контейнера с ботом).

Airflow vs Prefect vs Dagster: Для единственного разработчика **Prefect** часто предпочтительнее – он более простой, не требует отдельной БД/сервиса (можно использовать Prefect Cloud или запустить Prefect Server). Prefect позволяет писать workflows как обычный Python код с

декораторами, поддерживает динамические задачи (map) и легко интегрируется с Jupyter. Airflow – индустриальный стандарт с большим комьюнити, но сложнее в поднятии: нужен scheduler, workers, Postgres/Metastore ⁴⁶. Dagster – современный data orchestration, ориентирован на data pipelines и типизацию активов ⁴⁷. Он хорош для задач, тесно связанных с данными (например, ETL, ML), имеет сильную встроенную модель данных и UI (Dagit), но порог входа выше, чем у Prefect.

Failure semantics: - В Airflow при падении таска он может автоматически ретраить (по настройке `retries`). Но если упадёт сам scheduler – понадобится человеческое вмешательство. Зато все состояния хранятся в метаданне (БД) – можно продолжить. - В Prefect при использовании Cloud – автоматически ретраит согласно политике, агент помечает flow как failed или resumed. Prefect **реализует шаблоны circuit-breaker** – например, можно задать, что если 3 задачи подряд упали, то flow прекращается ⁴⁸. - Dagster схож с Prefect по возможностям retries, плюс у него есть встроенные сенсоры, отслеживающие появление новых данных.

Для наших нужд – Prefect 2.x выглядит оптимально: лёгкий, Pythonic, можно запускать флоу просто как скрипт. Например, запустить flow `data_ingest_flow()` раз в минуту с помощью Prefect Orion, или даже без центрального сервера, используя *LocalAgent*.

Мониторинг и метрики: Институциональный подход требует **observability** в реальном времени: - Все критические модули снабжены **Prometheus-метриками**: количество собранных тиков, задержка обработки (лаг между временем тикера и записью в БД), число активных позиций, текущий PnL, использование памяти/CPU, и т.д. Эти метрики экспонируются, например, через *HTTP endpoint* (можно интегрировать с FastAPI). Поднимаем Prometheus-сервер, собирающий метрики каждые 15 сек. - **Grafana**: настраиваются дашборды – график баланса и PnL в реальном времени, график числа ошибок API, latency WS, проскальзывание в сделках. Также алерты: Grafana/Prometheus может отправлять уведомления (в Telegram/Email) при событии – напр., «*Latency > 1s*» или «*Daily PnL < -3%*». - **Логирование и трассировка**: Логи (info/debug) всех модулей пишутся в ротационные файлы. Для удобства анализа можно добавить централизованный сбор – ELK (Elastic + Kibana) или более лёгкий стек (например, Grafana Loki). Но для начала, распределённых сервисов нет, можно обойтись просмотром локальных логов. Главное – формат логов должен включать timestamp, модуль, уровень, message. Например: `2025-10-21T14:00:00.123 [DataIngest] INFO Fetched 120 trades (BTC/USDT)`. - **Audit logs**: Отдельно сохраняются чувствительные действия: подключения к API, выставление ордеров, изменения настроек. Эти audit-логи могут понадобиться для комплаенса (налоги, разбор инцидентов). - **Модели дрейфа**: Если используются ML-модели, внедряется мониторинг *качества прогнозов*. Например, каждую неделю рассчитывать попадание модели: сколько сигналов были правильными. Метрика **обучающая/боевая эффективность** покажет дрейф модели. Также можно отслеживать статистику входных фич – средние, корреляции – и сигнализировать, если распределение сильно отклонилось от обучающего (это симптом, что модель устаревает). Для авт. обнаружения концептуального дрейфа есть алгоритмы (пример: ADDM – Autoregressive Drift Detection Method, который мониторит изменение стат. свойств серии ошибок модели ⁴⁹ ⁵⁰). Наш мониторинг может упрощённо: считать скользящее окно точности и, если оно падает ниже порога, выдавать алерт «*Model drift suspected – consider retraining*».

CI/CD и тестирование: Каждое изменение кода должно проходить через unit-тесты. Настраивается CI (например, GitHub Actions) для автоматического прогона тестов при пуше. В тест-матрицу входят: - Тесты модулей: сбор данных (эмулировать ответ биржи и проверить парсинг), расчёт индикаторов, генерация сигналов (на небольшом known dataset), исполнение (мокнуть API биржи и проверить логику OMS). - **Бэктест регрессии**: Для ключевых стратегий можно хранить эталонный результат бэктеста на фиксированном историческом периоде. После

изменений стратегии – прогонять снова и сравнивать метрики. Допустимые расхождения (например, +/-0.5% по финальному PnL). Если больше – значит, изменения повлияли, и нужно анализировать, было ли это ожидаемо. - **Data contract checks:** использовать, например, Great Expectations – написать ожидания, что датафрейм свечей имеет колонки [open, high,...], нет пустот, цена > 0, объём >=0. Запускать эти проверки как часть pipeline (особенно на новых данных). Это позволит поймать, если, скажем, биржа внезапно ввела новую колонку или изменила формат (API drift).

Deployment: Для доставки обновлений можно использовать Docker + Docker Compose. Тестирование проводить на staging-конфигурации (например, paper trading на тестовой бирже или с ключами на минимальных балансах). После успешных испытаний – деплой на прод. Стоит обеспечить возможность *быстрого отката (rollback)*: хранить предыдущую стабильную версию образа, и если новая дала сбой – мгновенно вернуть старую. Можно применить blue-green deployment: запустить новую копию бота параллельно (в режиме только записи логов, без отправки ордеров) и сравнить поведение, прежде чем переключить. Но это для одной стратегии избыточно, скорее просто откат по команде.

Развертывание оркестрации: Если выбрали Prefect Cloud – CI/CD сводится к обновлению кода flows, Prefect сам возьмет новое. Если Airflow – можно настроить CI на деплой DAG-файлов и перезапуск scheduler. В Docker Compose можно отделить сервисы: *bot, db, grafana, prometheus*. Тогда обновление бота – заменить образ и `docker-compose up -d`.

Alerts и нотификации: Помимо метрик, настроить оповещения: интегрировать, например, с Telegram ботом, чтобы платформа присылала критичные события (ошибки подключений, срабатывание стоп-аута). Это важно, т.к. единственный инженер не может 24/7 смотреть графаны – а так в случае чего он проснётся от уведомления.

В сумме, через эти меры достигается **24/7 надёжность**: процессы автоматизированы (от сбора данных до отчётов), а наблюдаемость (мониторинг) позволяет быстро обнаружить и отреагировать на проблемы. Это особенно важно, ведь нет круглосуточной команды поддержки – все экстренные ситуации должен уметь обработать сам код или, в крайнем случае, быстро привлечь внимание инженера через алерт.

8. Безопасность и соответствие требованиям (Security & Compliance Basics)

API-ключи и доступы: Торговые ключи бирж – главная точка уязвимости. Принципы: - **Принцип минимальных привилегий:** ключи должны быть созданы с *минимальным набором прав* ⁵¹. Для нашего бота достаточно прав на чтение и торговлю (ордера). **Вывод средств отключается.** Никогда не храним ключ с разрешением Withdrawal – даже если злоумышленник украдёт ключ, он не сможет вывести деньги ⁵² ⁵³. (Пример: на Binance API-ключи можно явно задать разрешения и отключить выводы – надо использовать это). - **Белый список IP:** Биржи позволяют ограничить действие ключа определёнными IP. Нужно задать IP сервера, где бот стоит ⁵¹. Тогда если ключ утечёт и кто-то попытается с другого адреса – будет отказано. - **Хранение секретов:** Ключи не хардкодятся в код. Используем .env файл или хранилище секретов. Например, .env файл:

```
BINANCE_API_KEY="x123..."
BINANCE_API_SECRET="y456..."
```

```
OKX_API_KEY="..."
OKX_API_SECRET="..."
```

И т.д. Docker Compose при запуске будет считывать это и передавать в контейнеры как переменные окружения. Файл .env держать зашифрованным (например, использовать ansible-vault или простое шифрование когда хранится в репо), или не хранить в репозитории вовсе – а разворачивать вручную на сервере. Для ещё большей безопасности – интеграция с Hashicorp Vault: боту даётся токен доступа, и при старте он вытягивает ключи из Vault (который, в свою очередь, защищён). - **Регулярная ротация ключей:** раз в несколько месяцев можно обновлять API-ключи ⁵¹. Хотя это не всегда удобно (надо обновлять в конфиге), но снижает время жизни потенциально утекшего ключа. - **Шифрование в памяти:** Если пишем логин/пароль от чего-то, то в памяти можно держать недолго. Но в нашем случае в основном API keys – их в запросах в любом случае нужно держать.

Сетевые и системные меры: Желательно запускать бота на изолированном VPS/сервере. Все ненужные порты закрыть (фаервол). Доступ – только по VPN или SSH ключам. На самом боте включить *read-only FS* для критичных частей (чтобы вредонос не смог подменить бинарники). Docker-контейнеры запускать без лишних привилегий.

Защита от DDoS/Bad data: Бот полагается на внешние API – если злоумышленник сумеет подsunуть ему ложные данные (например, через манипуляцию DNS или MITM), могут быть неверные решения. Поэтому: - Включить SSL verification (HTTPS) для всех запросов (по умолчанию да, но проверить настройки). - Желательно фиксировать IP бирж (через `/etc/hosts` или async DNS с pinning), чтобы не подхватить подмену. - Проверять, что данные от API реалистичные (например, если пришла цена BTC = \$1, это явно сбой – игнорировать).

Логирование и соответствие комплаенс: Даже если это личный бот, стоит вести логи, удовлетворяющие требованиям налоговых и аудита: журнал сделок с точным timestamp, цена, количество, комиссия. Потом на основе этого проще считать налоги (особенно если множественные сделки, нужен FIFO расчет прибыли). Желательно уметь экспортировать историю торговли в формат, удобный бухгалтерскому учёту (CSV, Excel). Также, если планируется привлечение внешнего капитала, такая прозрачность поможет пройти due diligence.

Синхронизация времени (ещё раз): Помимо NTP, некоторые биржи (OKX, Binance) предоставляют метод `/time` – можно при запуске бота сравнить отклонение локального времени и серверного. Если > 100ms, залогировать предупреждение. В критичных системах устанавливают PTP (*Precision Time Protocol*), но для нас NTP достаточно.

Бэкапы и аварийное восстановление: Необходимо периодически делать backup важнейших данных: база (если Timescale – дампы раз в день, или реплика). Логи – тоже, по крайней мере копировать off-site (например, на облачный диск раз в неделю). В случае фатального сбоя сервера – должна быть процедура DR: - Иметь готовый резервный сервер (или возможность быстро развернуть Docker-окружение на новом). - Шаги восстановления: установить docker/docker-compose, поднять контейнер БД (развернув из последнего бэкапа), затем контейнер бота с последним стабильным образом, применить .env с ключами – и запустить. Этот **runbook** восстановление описывается и хранится где-то открыто для инженера. В стрессовой ситуации (ночью сервер сгорел) – по инструкции можно быстро всё поднять. - Регулярно тестировать восстановление на staging: имитировать потерю сервера и разворачивать заново на другом – это выявит, не упущены ли какие-то детали (например, что нужно не забыть открыть порт 9100 для Prometheus, и т.п.).

Разграничение доступа и привилегий: Если платформа развёрнута в облаке, следовать принципу *least privilege* и там: учётные записи облака, сервис-аккаунты имеют только нужные права (например, EC2 instance role только на чтение CloudWatch, если нужно). Пароли доступа к интерфейсам (например, Grafana) задать сложные и уникальные. Хотя это личная система, но лучше претворять практики как для фирмы.

Соответствие требованиям: Если объёмы начнут быть существенными, потенциально возникают требования регуляторов (например, торговля деривативами – отчётность). Также, если планируется подключение стороннего капитала – нужна будет отчётность перед инвестором. Заложим, что система может генерировать месячный отчёт: список сделок, PnL, метрики риска – в формате PDF/Excel.

Противодействие сбоям биржи: Безопасность включает не только киберриски, но и форс-мажор: биржа может лечь (бывали случаи, что Binance висел часами). Бот должен распознавать такое – если при запросах timeout или ошибки уровня 500 – это проблема биржи. Стратегия: либо переключиться на запасную биржу (если есть), либо остановить торговлю до восстановления. Нельзя слепо продолжать посылать приказы в пустоту.

Документация и проверка: В целях комплаенса вся конфигурация (ключи, настройки риска) документируется. В продакшене институциональном проводят *penetration testing*, у нас можно хотя бы static code analysis (линтеры) запускать, чтобы отловить потенциальные уязвимости.

В целом, подход: считать платформу **Production-Ready** с точки зрения безопасности – это включает дисциплину управления ключами ⁵¹ ⁵⁴, логирование для аудита и план готовности к инцидентам. Эти меры минимизируют риск как финансовых потерь от кражи/API ошибок, так и юридических проблем (например, для налогов все сделки подкреплены журналами).

9. План внедрения (Implementation Plan)

Этапы разработки: Предлагается разбить реализацию на этапы с наращиванием функционала:

- **MVP (4–6 недель):** Базовая рабочая система:
- Сбор данных для 1–2 инструментов (например, BTC/USDT) с одной биржи (Binance) – через CCXT. Хранение в простой форме (CSV или SQLite/Parquet) для начального этапа.
- Реализация 1-2 индикаторов и простой стратегии (пересечение MA или тренд по RSI).
- Простенький бэкtest этой стратегии на исторических данных – убедиться в работоспособности логики.
- Модуль исполнения: подключение к бирже с тестовыми ключами (или Binance Testnet), размещение реальных ордеров минимальным лотом. Провести пробную «торговлю» на небольшом капитале или paper trading.
- Минимальный риск-менеджмент: стоп-лосс на каждую сделку, лимит 1% капитала на сделку.
- Логирование в консоль/файл, базовые алерты (например, при ошибке API – отправить email).

Acceptance Criteria MVP: Платформа способна собрать свечи и трейды, записать их, сгенерировать сигнал и исполнить его на бирже. Получен первый отчет о торговле (пусть даже на тестнете). Все

компоненты работают в течение 24ч без вмешательства (пройден smoke-test 24/7). Документирована инструкция запуска.

- **Beta (8-12 недель):** Расширение функционала и надежности:
- Подключение ещё 1-2 бирж (OKX, Coinbase) через модульный интерфейс. Унификация через CCXT.
- Перенос хранения в **TimescaleDB** или PostgreSQL для боевого режима. Импорт уже собранных данных в БД.
- Добавление 5-10 технических индикаторов (TA-Lib) и возможно простого ML-модуля (напр., логистическая регрессия на фичах).
- Валидация стратегии: провести walk-forward тест за последние 1-2 года, параметры подобрать. Оптимизация: возможно применить optuna для подбора параметров индикаторов.
- Реализовать основные риск-контролли (daily stop, ограничение плеча) и позиционирование (фиксированный %).
- Оркестрация: внедрить Prefect или Airflow для планирования ежедневных задач (например, расчет отчета в полночь, пересборки модели по понедельникам).
- Monitoring stack: поднять Prometheus и Grafana, настроить метрики PnL, latency, ошибок. Настроить алерты в Telegram.
- Безопасность: перевести ключи в env, включить whitelist IP на биржах, протестировать kill-switch сценарий (например, имитировать превышение лимита).
- Документация: описать все конфиги, добавить README для запуска нового окружения.

Acceptance Criteria Beta: Система работает с несколькими биржами и инструментами параллельно. При потере соединения автоматически восстанавливается и догружает данные. Мониторинг показывает метрики, нет memory leak (работает >1 неделе стабильно). Стратегия на демо-аккаунте не нарушает лимитов (проверено, что стопы срабатывают). Получен стабильный результат бэктеста, который принят как целевой для прод (например, стратегия X с Sharpe ~1.5 на 3 года истории).

- **Production Hardening (к 4-6 месяцу):** Доводка до индустриального уровня:
- Развёртывание на надёжной инфраструктуре (например, VPS в датацентре с UPS, либо облако AWS/Google). Настройка резервирования БД (реплика или регулярные бэкапы).
- Оптимизация производительности: профилировать узкие места (возможно, переписать критичные расчёты на NumPy/Numba/Cython). Например, если сбор тиков отстаёт, можно распараллелить по инструментам или перейти на асинхронный сбор.
- Масштабирование: добавить оставшиеся желаемые рынки (в сумме, например, 10 пар на 3 биржах). Проверить, что БД справляется (Timescale легко держит миллионы строк ¹⁶, но убедиться в скорости).
- Full risk management: добавить расчет VaR/EVaR для портфеля в реальном времени, автоснижение позиций при росте волатильности (элемент адаптивного риска).
- Провести тестирование «грязных сценариев»: падение биржи – бот корректно останавливается, переподключается; экстремальный рывок цены – срабатывают стоп-лоссы, потери ограничены.
- UI/Reporting: возможно, прикрутить простой дашборд (на базе FastAPI + React или Grafana dashboards) для визуализации баланса, активных позиций, PnL. Автоматический генератор отчёта (таблица сделок за месяц, показатели) – экспорт PDF.
- Compliance: подготовить шаблон отчёта для налоговой с прибылями по каждой сделке (если нужно).

- **Финальное тестирование:** запустить систему на небольшом реальном капитале (например, \$1000) на 1 месяц. По итогам сравнить результаты с ожидаемыми, отладить последние нюансы.

Acceptance Criteria Prod: Система выдержала непрерывную работу 1 месяц с реальными торгами без重大 сбоев. Все потери укладываются в оговоренные лимиты, несанкционированных позиций/ордеров не зафиксировано. Инженер может **в один клик развернуть аналогичный экземпляр** (благодаря Docker и документации). Платформа готова к расширению (можно добавить новую стратегию или актив, не ломая существующий код).

График и ресурсы: Предполагая ~10-15 часов в неделю разработки, MVP за ~1 месяц, Beta за ~2-3 месяца, прод через ~4-5 месяцев. Бюджет трудозатрат: MVP – 100 часов, Beta + Prod – ещё ~200-300 часов (включая тестирование, доку, улучшения). Финансово: поначалу можно пользоваться бесплатными сервисами (free tier Prefect Cloud, бесплатный TimescaleDB instance до 10M строк, маленький VPS). На этапе прод будет небольшой операционный бюджет (см. ниже).

Риски планирования: Возможны задержки из-за отладки биржевых тонкостей (каждая API своя) и доработки стратегии (может потребоваться пересмотреть подход, если тесты покажут несостоятельность). Закладывается резерв времени на перекалибровку стратегии после Beta-тестов.

Roll-back plan: При выпуске новой версии бота всегда держать предыдущую: если новая вызывает хаос (неправильные ордера), автоматически переключиться обратно. Например, можно версионировать Docker images: `quantbot:v1` (стабильный), `quantbot:v2` (кандидат). Запустить их параллельно (второй в dry-run режиме). Только убедившись, что v2 даёт те же сигналы, заменить. Если всё же на проде что-то пошло не так – за секунды остановить v2 и поднять v1. Все это можно автоматизировать (скрипт деплоя с откатом).

Тестовый список (Test Matrix): - Юнит-тесты: функции расчёта индикаторов, функция выставления ордера (с моком API), логика `risk_check` (например, проверка, что при `position_limit = 0` не открывает новую). - Интеграционный тест: запустить бэктест на 1 день данных и сопоставить с ручным вычислением результата. - Тест нагрузки: смулировать быстрый поток (напр. 1000 тиков/сек) через генератор – проверить, что ingestion/обработка успевает (без потери). - Failover тест: убить процесс БД и поднять заново – бот должен переподключиться (или хотя бы не рухнуть безвозвратно). - Security test: использовать какой-нибудь сканер на зависимости (проверить, нет ли уязвимостей в используемых библиотеках). - User Acceptance Test: если конечный «заказчик» – сам инженер, то фактически критерий – удобно ли ему с этим работать? Это проверяется на этапе Beta: насколько легко добавить ещё индикатор, легко ли смотреть отчёты? Отладить DX (developer experience).

Бюджетные оценки (ежемесячно): - *Облако (сервер):* Можно начать с VPS 2 CPU 4GB RAM – ~\$20/мес. Если использовать AWS, то EC2 t3.medium ~\$40. Для высоконадежного – 2 ноды по \$40 (primary+failover). На этапе прод бюджет ~\$100/мес на вычисления. - *Хранилище:* TimescaleDB можно хостить на том же сервере (экономия), или взять managed Timescale Cloud (дороже, минимум ~\$200/мес за продвинутый план). Пока объёмы невелики, локальный DB ок. Можно заложить ~\$20/мес (SSD диск 100GB). ClickHouse, если понадобится, скорее всего self-host (учитываем CPU/RAM в основном сервере). - *Мониторинг:* Prometheus+Grafana – open source, развёрнутые на том же сервере. Стоимость – по сути, хранение метрик (пара гигабайт места) и нагрузка CPU ~5%. Так что отдельно \$0 (в рамках сервера). Если использовать облачные (Datadog, etc) – они платные, но не требуются. - *Сетевые расходы:* трафик небольшой (пара ГБ в месяц), обычно включено в сервер. - *Итого минимально:* ~\$50-100/мес при self-host. **Средний уровень:**

Если вынести БД и мониторинг в managed: БД \$200, сервер \$40, итого ~\$240. Добавим запас – **\$250-300/мес.** - *Максимальный (с резервами):* 2 сервера x \$50 + Managed DB cluster \$300 + Managed alerting \$50 = ~\$450-500/мес.

Эти уровни можно масштабировать в зависимости от успеха: начать с минимального, по мере роста капитала инвестировать в надежность.

10. Обзор кейсов и бенчмарков (Case Studies & Benchmarks)

При проектировании учтены существующие решения и исследования:

1) Freqtrade (Open-Source Bot) ⁵⁵ : свободный крипто-бот на Python. Архитектура – монолитный, управляется через CLI/веб. Поддерживает множество бирж через CCXT, стратегию пользователь пишет на Python классе (с индикаторами, etc). Плюсы: хорошая документация, активное сообщество, реализованы основные функции (от бэктеста до оптимизации). Минусы: ориентирован на *стратегии на свечах* – т.е. не event-driven внутрисвечной трейдинг. Также, риск-менеджмент там базовый (стопы, капитал на пару). **Reuse:** можно взять идеи по структуре проекта (например, как подключаются биржи, как хранит конфиги). **Avoid:** не будем копировать их генератор стратегий (у нас свои), и, возможно, не подойдет их ORM/БД – они используют SQLite/JSON для простоты, что не *institution-grade*. Но как пример, Freqtrade подтвердил жизнеспособность подхода: один бот, многие биржи, на Python, для умеренных страт – работает.

2) Hummingbot (Market Making) ⁵⁶ : фреймворк с фокусом на маркет-мейкинге. Написан на Python, но модульный (разделены коннекторы бирж, стратегии, скрипты). В Hummingbot продумана система ордеров – он умеет держать одновременно заявки на покупку и продажу, переставлять их. Плюс поддерживает высокую частоту (для Python – многопоточно/асинхронно). **Reuse:** можно взглянуть на их реализацию коннекторов – например, обработка WebSocket, модель ордера, контроль задержек. Возможно, использовать их код в качестве библиотеки? (Hummingbot имеет API для стратегий). **Avoid:** Hummingbot довольно сложен в конфигурации, и у него своя консоль. Для нашей цели – избыточно, т.к. мы не строим HFT-мейкинг. Но идеи по **реализации post-only, выстраиванию лестницы ордеров** – возьмём. Отметим, Hummingbot широко используется маркет-мейкерами, значит, его ядро надёжно. Но оно не предназначено для произвольных стратегий (нет бэкестинга, фичи ML и т.д.).

3) NautilusTrader (Open Core Trading Platform) ⁵⁷ ⁵⁸ : позиционируется как “самая быстрая open-source платформа”. Пишется на Python с критическими частями на Rust для скорости. Поддерживает множественные классы активов и *event-driven backtest*. Упор на качество исполнения и надежность. **Insights:** Nautilus подтвердила выбор event-driven подхода – они явно рекламируют *low-latency event simulation* ⁵⁸ ⁵⁹ . Также, они делают *live trading без изменений кода после бэктеста* – то же наш принцип (чтобы стратегия, проверенная на истории, точно так же работала в реале). **Reuse vs Avoid:** Код Nautilus открыт частично (open-core). Можно изучить их структуры данных для ордербуков, возможно, заимствовать идеи по интерфейсам (создание стратегии как класс, подключение data feeds). Однако тянуть всю Nautilus не будем – она ещё в развитии и может быть избыточна. Но, например, можно воспользоваться их модулем *nautilus-crypto* для подключения бирж, если он зрелый. В то же время, самописный более контролируем. И Nautilus, будучи высокопроизводительной, предполагает, что пользователь хорошо разбирается – для нас, одного человека, лишний complexity cost.

4) Backtrader vs VectorBT vs Zipline (сравнение) ⁶⁰ ³⁵ : Согласно опыту комьюнити, - *Backtrader* – лучший для простоты и live-трейдинга у ретейла, но не масштабируется ⁶¹ ⁶² . - *Zipline/*

Quantopian (устаревшая) – была академична, но сейчас практически заменена *QuantConnect*. - *VectorBT* – “blazing speed, large data, portfolio strategies” – сообщество отмечает, что **для систематика VectorBT трудно превзойти по скорости и возможностям** ³⁵. В нашем дизайне мы уже решили использовать *VectorBT* для исследовательских задач. - *Lean (QuantConnect)* – open-source, но на C#. Он хорош для масштабных бэктестов и multi-asset, однако для self-host требовал существенных усилий (поднять всю инфраструктуру QC). В 2023 они упростили self-host, но всё равно – более чужеродный стек (C#, Docker, Linux). Для Python-разработчика, что мы, *Lean* – возможно, излишний. Тем не менее, можно взять *концепции Lean*: например, у них есть **Transaction Manager** с просчитыванием fill на основе моделей ликвидности – такие идеи можем реализовать упрощённо.

5) Рынок данных: Kaiko, CoinAPI, CryptoCompare: - *Kaiko* – институциональный провайдер данных (трейды, книги, индексы) с высокими ценами, но и высокое качество. Для личного пользования, возможно, overkill, но знать стоит. Их кейс: если понадобится исторические тики за много лет – купить у *Kaiko* пакет, вместо сбора самому (экономия времени). - *CoinAPI* – унифицированный API данных. Удобен для исторических данных: например, можно одним запросом выкачать все свечи с 2017. Но он платный с ежемесячной подпиской. - *CryptoCompare* – даёт бесплатные и платные API по историческим и текущим ценам (с ограничениями частоты). Можно использовать как резервный источник (например, если основной сборник отстал, догрузить с *CryptoCompare*). - *CoinGecko* – бесплатный API, но **только справочная и агрегированная информация** (цены 1 минутка он не даёт, только более крупные интервалы, и данные по монетам/капитализации). Его применим для: список активов, символов, может быть, курс USD/Tether. **Вывод:** На этапе прод, если нужна более надёжная data feed, можно рассмотреть подписку на *CoinAPI* или др. Однако большинство бирж сами дают достаточно данных бесплатно через API – мы постараемся использовать их (плюс резервные публичные endpoints наподобие *Binance public data*).

6) ML и научные статьи: Обзор SSRN-публикаций ⁶³ ²⁰ показал актуальность ML-стратегий. В частности: - Интеграция on-chain и соц. данных (NLP по твиттеру) – это для future work, но платформа должна позволять добавлять новые data sources (например, через модуль фич). То есть дизайн учтёт, что не только рыночные данные могут приходить, но и CSV с sentiment – их тоже обработать. - Управление моделями: здесь можно взять пример **MLflow** usage как у OpenAI/Deer setups, но для начала, вручную тоже можно.

7) Docker & Infra: Практика показывает (пример – блоги Cloudflare про Timescale ⁶⁴) – даже крупные фирмы строят аналитические БД на Postgres/Timescale из-за удобства. Мы следуем такой же pragmatic стратегии, а не гонимся за exotic tech.

Что переиспользуем: - CCXT для подключения бирж – это стандарт де-факто, нет смысла писать с нуля все REST запросы. CCXT ⁵⁵ покрывает и *Binance*, и *OKX*, и *Coinbase*. Возможно, не будем использовать CCXT WebSocket (он в Pro версии), а возьмём например *websockets* + CCXT для REST. - Библиотеки TA (*pandas_ta*) – зачем реализовать RSI самому, если библиотека даёт точно проверенный? Скорость тоже приличная (TA-Lib на C). - *VectorBT* для backtest – его и возьмём. Это экономит огромное время на написание собственного движка, и даёт гибкость (вплоть до event-driven, если нужно). - **Докеризация и Compose:** будем использовать официальный образ *Timescale* или *Postgres*, возможно, официальный образ *Prometheus/Grafana*. Это всё reuse стандартного. - **Код из примеров** (GitHub johnbdfilio) – можно взять решение хранения *Parquet*, но у нас ВД, поэтому не надо. Но логика *asyncio* на сборе данных – там можно посмотреть как он сделал синхронный сбор. Мы likely сделаем свой, но зная, что концепция уже проверена – concurrency *asyncio yield from*.

Чего избегаем/уроки: - Опыт Quantopian/Zipline: шикарный бэкtest, но слишком комплексный pipeline (наследование class Algorithm). После их закрытия стало ясно, что closed ecosystem – риск. Поэтому наш код – open source компоненты и собственный простой код, без vendor lock. - Freqtrade UI: у них Telegram контроль. Возможно, нам не нужно пока, т.к. 24/7 бот автономен. Но на будущее – можно тоже добавить Telegram команды (например, остановись, дай статус). - Big Data trap: некоторые пытаются ставить Hadoop/Spark для финансовых данных. В нашем масштабе (tick data for 10-20 symbols) – достаточно ClickHouse. Не будем усложнять big-data технологиями, пока объем не превысит сотни млн строк.

Benchmarking / Performance: - По БД мы уже выполнили сравнение Timescale vs ClickHouse на основе RTABench ¹⁶: Timescale лучше на селективных запросах, CH – на full-scan агрегациях. - *Orchestrators*: Приведём сводную таблицу ниже, суммируя наши находки. - *Backtest frameworks*: Сводная таблица по VectorBT, Backtrader, Lean – их особенности, как они с комиссиями.

Сравнительная таблица: Базы данных для таймсерий

Характеристика	TimescaleDB (Postgres)	ClickHouse	DuckDB + Parquet	PostgreSQL (va
Запись (ингест)	До ~50k строк/с на одном узле (с индексами). Гипертаблицы упрощают вставку. Транзакционно – не теряет при сбоях ⁶⁵ ⁶⁶ .	Очень высокая скорость вставки батчами (100k+/с) за счёт колоночного подхода. Для стрима мелких порций – менее эффективна без батчирования. Нет полноценных транзакций (может быть несогласованность при сбое) ¹⁴ .	Вставка путём дозаписи в Parquet – эффективно большими порциями. В реальном времени затруднительно (накопление в буфере и периодический flush). DuckDB вставки ~50k/с, но без сетевого сервера (локально).	~10k строк/с пр правильной настройке (batch insert, без внешних ключей) ⁶⁷ ⁶⁸ . Требуется партиционирование при больших объёмах (иначе индексы раздуваются). А гарантии.
Чтение/запросы	SQL, сложные JOIN и агрегаты поддерживаются. Оптимизирован для WHERE по времени, <i>time_bucket</i> . Скорость высокая на интервалах, но уступает CH на полном скане. На RTABench ~1.9x быстрее CH ¹⁶ (т.к. нормализовано и фильтрация).	SQL-подобный (ClickHouse dialect). Максимальная скорость на агрегациях по всей таблице (анализ годов тиков). Может быть 5-10x быстрее Postgres на больших объёмах ¹⁶ . JOIN ограничены.	DuckDB SQL очень быстрый на Parquet (почти как CH на одном потоке). Хорош для аналитики ad-hoc. Но работает локально (чтение данных в RAM из файла каждый запрос).	Полный SQL, но аналитические запросы (много строк) медленнее 5-10 раз vs колоночные БД. С индексами хорошо отвечает точечные и мелкие выборки.

Характеристика	TimescaleDB (Postgres)	ClickHouse	DuckDB + Parquet	PostgreSQL (va
Партиционирование	Авто-партиции (chunks) по времени, с индексами (BRIN) для старых данных. Можно включить сжатие старых chunk (в 10-100 раз экономия) ⁶⁹ ⁷⁰ .	Явно задаётся <code>PARTITION BY</code> (например, месяц) + можно шардировать по ключу. Требуется немного ручной настройки, но потом работает прозрачно.	Parquet-файлы могут лежать по папкам (год/месяц/день) – и запрос DuckDB по маске путей даст фильтрацию по партициям. Ручное управление (удаление файлов для очистки).	Таблицы можно разделить по наследованию/секционированию (в PG10+ есть declarative partitioning). Но управление в ручную или через ограничения. Б. партиций – деградация на миллиард+ строк.
TTL (ретеншн)	Политики Retention и Drop chunks – удаление старых данных автоматом (фоновое). Можно настроить, напр., хранить 6 мес. подробно, старше – агрегаты.	TTL для мердж-деревьев: можно указать <code>TTL 30 DAY DELETE</code> при создании таблицы. CH сам будет удалять строки старше 30 дней.	Удаление = удалить файл или перезаписать Parquet без старых строк. Это дорогая операция (пересохранить). Обычно, просто храним все Parquet или вручную выпиливаем файлы старше X.	Вручную – либо DROP PARTITION, либо DELETE + VACUUM (что медленно). Для больших данных придётся регулярно чистить старые партиции (например, отцеплять таблицы старше года).
Стоимость и DevOps	Open-source, можно на одном сервере. Требуется настроить Postgres (SQL знать). Есть Managed Timescale (дорого: с ≈\$300/мес). Самому – DevOps умеренный (бэкапы pg_dump).	Open-source. Настройка немного сложнее (свой конфиг, мониторинг). Шардирование требует несколько инстансов. Managed ClickHouse (Altinity.Cloud и пр.) – \$\$\$. Самостоятельно – придётся следить за ресурсами, так как нет встроенного flow control (можно уронить при сверхнагрузке).	Полностью бесплатен. Очень прост в развертывании (библиотека). Нет отдельного сервиса – легче деплой/тест. Однако, нет удалённого доступа – либо запускать на той же машине, либо городить свой сервер вокруг. Бэкап – просто копировать Parquet файлы.	Бесплатен. Широко известен администраторам. Но при больших нагрузках придётся тюнить (shared_buffers, ...). Требуется большое ручное внимание при росте объёма индексов, partition maintenance. Лучше найти DBA наёмно при необходимости.

Характеристика	TimescaleDB (Postgres)	ClickHouse	DuckDB + Parquet	PostgreSQL (va
Применение	Хорош для трейдинговой истории, где нужны сложные запросы, комбинирование с другими данными (например, on-chain). Отличный компромисс скорости/функциональность. Используется в отчетах (Cloudflare, etc) ⁷¹ .	Идеален для аналитики тиков, хранения <i>сырых</i> лент больших объёмов. Если нужен хранилище под BI на миллиардах строк – CH. Для live-трейдинга используется реже из-за eventual consistency, но возможно (нужно внимательно делать upsert).	Подходит для локального анализа, исследований. В проде – разве что для офлайн-аналитики или бэктестов. Можно использовать как <i>feature store</i> на файловом уровне. Не подходит для многопользовательской работы.	Хорош для умеренных объёмов (сотни миллионов строк) при хороших серверах. Использовать, не хочется новую технологию – не придётся мириться с более низкой скоростью аналитики. Для мелких историй (<50М строк) Postgres вполне справится и при чем интегрирован с другой движок.

Сравнение оркестраторов: Airflow vs Dagster vs Prefect

Критерий	Apache Airflow	Dagster	Prefect 2
Архитектура	Централизованный scheduler + workers, метаданные в БД ⁴⁶ . DAG-файлы (Python) разворачиваются на сервере. Масштабируется via Celery/K8s Executor.	Разделяет планировщик (Daemon) и пользовательский код (грузится через gRPC) ⁴⁷ . Чёткая модель <i>Assets/ops</i> с типизацией и зависимостями. Web-UI (Dagit) для наблюдения.	Агент + Flow. Оркестрация может в Prefect Cloud (SaaS) или локально (Prefect Orion) ⁷² . Агент постоянно опрашивает очереди и запускает задачи как процессы/контейнеры – нет постоянно тяжёлого scheduler.

Критерий	Apache Airflow	Dagster	Prefect 2
Developer Experience	Декларативные DAG: определяются через Python код, но много шаблонного кода (XCom, context). Кривая обучения средняя ⁷³ . Огромная экосистема операторов (2000+) ⁷⁴ , но качество операторов разнится. Требуется знание Jinja темплейтов при сложных задачах.	Современный подход: задачи описываются как функции с декоратором <code>@op</code> или <code>@asset</code> . Имеет первую гражданскую единицу – <i>Assets</i> (данные), что обеспечивает автоматическое построение lineage. Типизация через Python type hints, интеграция с pytest для тестов ⁷³ ⁷⁵ . Порог входа выше Airflow, но предоставляет богатые возможности разработки (time-travel дебаг, <i>software-defined assets</i>).	Максимально простой стиль: писать обычный Python код. Задачи – функции с декоратором <code>@task</code> , которые можно <code>task.map()</code> . Поддерживает императивные конструкции (loops), что сложно в Airflow. Логи стримятся в UI Prefect Cloud в реальном времени ⁷⁶ . Очень низкий входной барьер – “просто Python”.

Критерий	Apache Airflow	Dagster	Prefect 2
Надёжность и наблюдаемость	Зрелый проект: множество кейсов, но monitoring встроен скромно (UI показывает статус DAG, но метрики и алерты надо настраивать отдельно – через StatsD, Prometheus экспортер). Имеет SLA-мониторинг, но зачастую фирма внедряет дополнительный мониторинг. При сбое scheduler или воркера – возможно, потребует ручного вмешательства.	С самого начала ориентирован на <i>observability</i>: отслеживает события, версионизирует данные (asset materialization). Имеет логирование каждого события в богатом виде. Можно проще отлаживать, т.к. Dagster UI позволяет запускать шаги. Failure handling: можно задать сквозные ретраи, или отлов определённых ошибок – гибко.	Prefect Cloud отображает все запуски и состояния. Автоматически применяет retry-политики, паттерны отключения (например, можно настроить, что если 3 задачи флоу упали, он не будет дальше исполнять) ⁴⁸. Есть нативные уведомления (state handlers). Преимущество – агент работает независимо: даже если сервер (cloud) недоступен, запущенные flow могут завершиться.
Интеграции	Огромная библиотека операторов: от AWS, GCP до SSH, Spark. Практически любой сервис уже есть plugin. Это упрощает создание DAG (вызвал готовый оператор). В новых версиях поддержка Docker, Kubernetes pod, etc.	Имеются готовые библиотеки (dagster-snowflake, dagster-dbt и др.) ⁷⁷, но их меньше, чем airflow operators. Dagster больше рассчитан на Pythonic integration: т.е. проще обернуть свой код, чем тянуть внешний operator. Зато prefect/airflow операторы можно адаптировать.	Prefect предоставляет <i>Collections</i> (prefect-dbt, prefect-kubernetes, etc) – добавляют задачи для популярных инструментов ⁷⁷ ⁷⁸. Но их меньше, чем у Airflow. Однако Prefect легко запускать shell-команды или писать небольшой Python, так что отсутствие готового оператора не больно.

Критерий	Apache Airflow	Dagster	Prefect 2
Производительность и масштабирование	Airflow 2.x улучшил планировщик (100+ DAG/s). Масштабируется горизонтально: можно запустить десятки воркеров, разнести scheduler и UI. Однако большой DAG с тысячей задач может долго парситься и съесть память (т.к. DAG-файл исполняется при каждом scheduler heartbeat) ⁷⁹. Есть ограничения на динамические таски (не более ~1024).	Dagster изолирует исполнение через gRPC, что снижает нагрузку на основной процесс ⁸⁰. Масштабирование – путем деплоя Dagster опсов на Kubernetes (Dagster Cloud облегчает это). В целом, способен держать тысячи assets, как заявляют, (у крупных пользователей – тысячи ежедневных задач) ⁸¹.	Prefect Agent модель – очень гибко: можете запустить 100 агентов на разных узлах, они будут брать задачи. Каждый flow run – отдельный процесс/контейнер, не мешает другим. Высокая параллельность – лишь ограничена ресурсами машин. В Cloud нет ограничений на кол-во flow (платишь за task-run). Для on-prem Prefect Orion – можно масштабировать Dask исполнители, но для наших целей хватит и sequential.
Стоимость	Open-source Apache 2 – бесплатно. Но нужно содержать инфраструктуру: VM для scheduler, БД (Postgres), возможно Celery broker (Redis). Есть managed Airflow (AWS MWAA, Astronomer) – но это от \$200/мес.	Open-source. Можно использовать Dagster Cloud (предлагает UI SaaS, а код у вас) – тарифы зависят от compute-часов. Самому хостить – бесплатно, но придётся поддерживать сервисы (Postgres для schedules, daemon).	Prefect OSS – бесплатно, но тогда без nice UI (хотя Orion UI локально тоже есть). Prefect Cloud – freemium (бесплатно до определённых лимитов). Платно – за определённое число task-run (например, \$0.25 за 10k runs), для небольшого проекта скорее \$0. Предпочтительно начать с Prefect Cloud бесплатного – нулевые затраты на хостинг.

Критерий	Apache Airflow	Dagster	Prefect 2
Когда использовать	Если уже знакомы с Airflow или нужно соответствие enterprise (многие фирмы уже имеют Airflow). Хорош для разнообразных задач (ETL, отчёты, ML) в одной платформе. Можно найти много готовых примеров.	Если важна надёжность data-пайплайнов и контроль данных. Например, вы генерите сводки, и нужно знать lineage. Dagster силён в командах data engineering, где код -> данные. Один разработчик может посчитать overkill, но если планируется рост – Dagster масштабируется.	Лучший выбор для старта одному разработчику. Минимум церемоний, можно быстро автоматизировать небольшие задачи (ингест, алерты). Prefect легко убирается, если станет мал – но практика показывает, что Prefect 2 хватает для 90% workflow. Особенно ценен для гибких и ad-hoc процессов, которые не строго DAG (if/else, loops).

Сравнение фреймворков бэктеста

Характеристика	VectorBT (Python)	Backtrader (Python)	Lean (QuantConnect)
Execution Model	Два режима: векторизованный (NumPy/Numba) и событийный (callback) ¹⁸. По умолчанию работает на массивах цен, генерируя ордера массивами – экстремально быстро, но без детального тикового порядка. Есть возможность event-driven: пользователь может писать колбэк on_bar или on_order для пошагового исполнения.	Event-driven backtester с покадровым обходом данных. Пользователь определяет метод <code>next()</code> в стратегии – он вызывается для каждой новой бар. Под капотом бэктестер проходит по данным в хронологическом порядке и вызывает стратегию, обрабатывая ордера. Скорость ниже, т.к. чистый Python на каждом шаге.	Событийный движок на C# (с Python support через CLI). Lean имитирует рынок вплоть до уровня matching engine: есть модели ордербука, стохастические задержки. Он проходит по тикам/барам, обрабатывает стратегию (на Python или C#), и исполняет ордера согласно моделям ликвидности/комиссий. Компилированный, высокопроизводительный (может прогонять годы истории tick-by-tick сравнительно быстро).

Характеристика	VectorBT (Python)	Backtrader (Python)	Lean (QuantConnect)
Fees & Slippage Support	Поддерживает простую модель: параметр <code>fees</code> или <code>slippage</code> при создании портфеля ¹⁸ ³⁰ – задаётся фикс. процент комиссии и фикс. проскальзывание в % или абсолют. Можно также вручную вычитать комиссии из PnL постфактум. Нет встроенного учёта <code>funding rate</code> – пользователь должен сам скорректировать цены или баланс (например, уменьшать баланс ежедневно на сумму финансирования).	Комиссии: есть встроенные классы <code>CommissionInfo</code> – можно задать процент от сделки или фикс. Backtrader при расчёте PnL вычитает их. Проскальзывание: можно подключить свою функцию <code>slippage</code> (например, моделировать цену исполнения). В базовой поставке есть простая модель “пункты проскальзывания”. Funding: отсутствует из коробки – можно реализовать как стоящийся “<code>commission</code>” или отдельный циклический эвент, корректирующий баланс (не тривиально, но можно).	Fees/slippage: Из коробки содержит модели для разных рынков. Например, для Binance у них реализована модель <code>fee = 0.1%</code> и проскальзывание на основе объёма сделки (по данным ордербука) ³³. Lean проводит симуляцию: если ордер слишком большой для исторического объёма, он частично исполняет. Funding: QuantConnect поддерживает загрузку дневных <code>funding rates</code> (в Lean <code>DataFormat</code>) и применяет их – позиция подвергается финансированию, как и реальная (в примерах есть решение с <code>adjust leverage</code>). Это близко к институциональной точности, но потребует исторических данных по <code>funding</code>.

Характеристика	VectorBT (Python)	Backtrader (Python)	Lean (QuantConnect)
Data/Asset support	Исторические данные подаются как pandas Series/DataFrame. Поддерживаются несколько колонок (multi-asset) – VectorBT позволяет портфель из множества символов. Но <i>внимание:</i> векторизованное тестирование предполагает синхронные временные индексы. Асинхронные события (разные торговые календари) сложнее, но решаемо (через event-driven режим).	Работает с <i>Data Feeds</i>: может быть CSV, Pandas, или live feed. Multi-asset поддерживается – стратегия может получать несколько data feeds (например, BTC и ETH). Backtrader синхронизирует их по времени (пропуская отсут. бары). Активы: акции, форекс, крипто – любые, формат данных OHLCV. Нет встроенной поддержки опционов/гreek.	Полная поддержка множества классов активов: акции, фьючерсы, опционы, крипто, форекс. Есть гигантская библиотека historical data (если пользоваться QC Cloud). Lean DataFormat – специальный формат хранения (который можно конвертировать из CSV). Multi-asset без проблем, синхронизация по самописному алгоритму. Опционы, фьючерсы – есть модели для экспираций, премий, margin. Это институциональный охват, но требует понимания каждого класса (не все задействуем).

Характеристика	VectorBT (Python)	Backtrader (Python)	Lean (QuantConnect)
Pros (Strengths)	Скорость: миллионы баров прогоняются мгновенно благодаря NumPy/ Numba оптимизации ¹⁸ . Можно делать параметрические исследования (перебор параметров) очень легко – он сам vectorize strategy by parameters. Интерактивность: интегрируется с Jupyter, есть методы для графиков, статистик – быстро итерации ресерча. Поддержка портфелей: удобные методы для ребалансировок, оценки метрик на портфельном уровне (например, max drawdown портфеля).	Простота использования: интуитивный ООП интерфейс, много примеров, огромное комьюнити (форумы, статьи). Гибкость стратегии: можно комбинировать несколько таймфреймов, несколько датасетов – Backtrader это умеет (менее прямолинейно в VectorBT). Live trading: главный плюс – стратегия после теста может быть запущена на брокера (IB, CCXT и т.д.) почти без изменений, т.к. Backtrader имеет live feeds. Это делает его универсальным “research-to-live” инструментом для розницы.	Реализм и масштаб: Lean – промышленный движок: учитывает нюансы исполнения, имеет встроенный Risk Management (как модули в QC). Мульти-стратегии: можно запускать несколько стратегий параллельно (портфель из стратегий). Бэкэнд C# даёт производительность – можно симулировать high-frequency (с субсекундным шагом) на большом потоке, Lean справится. Экосистема QC – много алгоритмов open-source, можно учиться на них.

Характеристика	VectorBT (Python)	Backtrader (Python)	Lean (QuantConnect)
Cons (Limitations)	Порог входа: для сложных логик event-driven придётся выйти из уютного vectorized стиля и писать callbacks, что менее задокументировано. Память: при очень длинных рядах (миллионы тиков) потребление памяти высокое (нужно 64-bit floats хранить). Execution simplistic: нет встроенных order book simulation – т.е. если вам нужно проверить скальпинг на стакане, VectorBT сам не поддерживает (но можно кодировать логику).	Производительность: на 1-миллион свечей тест может идти минуты, а вектор. подход – секунды. Для многокритериальной оптимизации Backtrader не подходит (слишком долго). Поддержка и обновления: проект не очень активно обновляется в последние годы, Python 3.9+. Есть форки, но риск, что каких-то новых фич нет. Не модульный: ядро закрытое, трудно внести свои улучшения без форка.	Сложность развёртывания: self-host Lean – надо настроить контейнеры, .NET environment, что непривычно для Pythonиста. Ограничение языком: писать стратегии на Python можно, но со своими ограничениями (QuantConnect API). Придётся учить их специфичный API (Initialize, OnData events и т.д.). Зависимость от QC data: если не использовать их data, надо конвертировать своё в их формат – overhead. В общем, Lean оправдан для крупного масштабного применения, но не для всех.

Как видно, наш выбор – использовать **VectorBT для быстрого ресерча**, а элементы из Backtrader/Lean – для идей по live trading. Мы перепишем исполнение самостоятельно, т.к. нужно тонко подогнать под наши риски.

Чек-лист рисков (Risk Checklist)

- Потеря данных (Data gaps):** Поток данных может прерваться (сетевой сбой, рестарт). Без должной обработки это приведёт к пустотам в котировках, и потенциально к ошибочным сигналам. *Митигация:* механизм ресинхронизации и бэкфилла ⁹, регулярный мониторинг полноты (алерт, если нет обновления >N минут).
- Сбой синхронизации времени (Clock skew):** Рассинхрон часов сервера с биржей может вызвать ошибки аутентификации (отклонение запросов) или неверные таймстампы записей. *Митигация:* NTP-сервис, проверка отклонения времени при запуске ¹⁷.
- Изменение API (API drift):** Биржа обновила API (URL, формат), бот может начать получать ошибки или некорректные данные. *Митигация:* Версионирование API (следить за анонсами биржи), абстракция через библиотеку (CCXT обновляется под изменения), юнит-тесты JSON-ответов.
- Дрейф модели (Model drift):** С течением времени стратегия/модель теряет эффективность (например, сменилась волатильность рынка) ⁴⁹. *Митигация:* Мониторинг метрик стратегии (SR, winrate). Авто-перетренинг моделей по расписанию, триггеры на ухудшение (если 3 месяца убыток – отключить стратегию).

5. **Некорректный выход из позиции (Orphan position):** Бот думает, что позиции нет (например, после сбоя), а на бирже она есть – риск неконтролируемой позиции.
Митигация: Периодическая сверка позиций через API ⁵, хранение позиций в БД и сравнение. При обнаружении – немедленный алерт и закрытие через kill-switch.
6. **Пропущенное исполнение стоп-лосса:** В резком движении цены стоп-ордер может проскочить (цена ускочила и стоп не сработал или слипедж). *Митигация:* Использовать рыночные стопы (гарантируют исполнение), закладывать запас в риск-модель (предположить worst-case slip).
7. **Отключение биржи (Exchange outage):** Биржа ушла в оффлайн (пример: остановки Binance). Бот не может закрыть позиции или обновлять данные. *Митигация:* Иметь условие: если API недоступен > X минут, зафиксировать позицию по альтернативным способам (если есть на другой бирже, или через брокера). Либо установить по умолчанию более консервативные позиции (чтобы выдержать).
8. **Всплеск проскальзывания (Slippage spike):** В момент низкой ликвидности сделка исполнилась по сильно худшей цене, чем ожидалось (проскальзывание). *Митигация:* Ограничение на размер ордера относительно объёма рынка (например, не больше 10% от объёма 5-минутки), использование IOC – лучше частичный fill, чем огромный slippage ⁴¹.
9. **Funding rates (финансирование):** Неправильный учёт финансирования на фьючерсах может привести к незапланированным убыткам. *Митигация:* Включить расчёт funding PnL в модель (отслеживать ставки, как Lean делает ³²). Ограничивать держание позиции вокруг funding выплат (не держать слишком долго, если ставка негативна).
10. **Делистинг или форк актива:** Если актив снимают с торгов или форк – позиции могут зависнуть. *Митигация:* Иметь обновляемый список торгуемых пар. При объявлении делистинга – бот автоматически закрывает все позиции по нему до дедлайна.
11. **Сквозная задержка (Latency):** Задержка от получения сигнала до отправки ордера. В волатильном рынке за эти миллисекунды цена может уйти. *Митигация:* Максимально держать исполнение локально (не через лишние прокси). Отдавать приоритет WebSocket (меньше latency, чем polling). Добавить маржу к цене ордера (если покупаем – чуть выше текущей, чтобы точно наполниться).
12. **Расчёт индикаторов на дырах:** Если свечные данные неполны (скачок времени), индикаторы могут дать неверный вывод. *Митигация:* Признаки проверяют целостность интервала (например, если пропущены свечи – не выдавать сигнал). Либо экстраполировать/замораживать на период пропуска.
13. **Сетевая безопасность:** Взлом сервера или утечка ключей. *Митигация:* Ключи без права вывода ⁵², IP whitelist ⁵¹, сервер за firewall/VPN. Регулярные обновления ОС, мониторинг логов (fail2ban для SSH).
14. **Человеческая ошибка:** Инженер мог задать неверный параметр (например, риск-порог 50% вместо 5%). *Митигация:* sanity-check на критичные конфиги (при запуске если daily_stop > 20% – логировать предупреждение). Встроенные ограничения – например, не больше 10 ордеров в секунду, чтобы в случае бага не зафлудить API.
15. **Регуляторные риски:** Изменение законодательства (например, запрет деривативов в юрисдикции). *Митигация:* Следить за новостями, иметь план свернуть определённые рынки при необходимости. Логи и отчёты хранить для доказательства добросовестности.
16. **Неучтённые корп. события:** (Если торгуем не только крипто, но акции) – сплиты, дивиденды. В крипто тоже есть события (airdrop, смена контрактов). *Митигация:* Поддерживать актуальность справочной информации – подписаться на уведомления биржи, обновлять символы (Chain rebranding – помнить, что это может влиять).
17. **Выход за лимиты API:** Бот может превысить rate limit и получать баны. *Митигация:* CCXT обычно обрабатывает rate-limit (с задержкой). Если пишем сами WS, надо ограничить частоту запросов. Добавить буфер запросов + backoff при ответе 429.

18. **Двойное исполнение сигналов:** Возможна ситуация, когда из-за лагов один и тот же сигнал исполнится дважды (например, повторный запуск бота). *Митигация:* Идентифицировать сигналы уникально (timestamp, id) и хранить «уже исполнено». Либо при рестарте сверять последние сигналы с последними сделками (не выполнено ли уже).
19. **Отказ брокера принять ордер:** Бывает, что API отвечает ошибкой (например, рынок в only-reduce mode, или ордер слишком мал). *Митигация:* Анализ кода ошибок – если получили ошибку, логировать и пытаться исправить (если это “ордер мал” – увеличить до минимума и повторить). Если не удаётся – закрыть позицию/не входить, уведомить.
20. **Высокая нагрузка на систему:** Если стратегий/активов станет много, бот может не успевать. *Митигация:* Метрики мониторят задержку обработки. При превышении – выдавать алерт. Масштабировать: вынести сбор данных отдельно, торговлю отдельно (микросервисы). Увеличить ресурсы (CPU/RAM). Оптимизировать горячие точки (векторизация расчётов).

Этот чек-лист будет использоваться при тестировании (например, симитируем рассинхрон часов, посмотрим, ловит ли бот) и при написании процедур (например, disaster recovery).

Приложения (Appendices)

Пример .env (переменные окружения):

```
# API keys for exchanges
BINANCE_API_KEY="live-binance-key-here"
BINANCE_API_SECRET="live-binance-secret-here"
OKX_API_KEY="okx-key"
OKX_API_SECRET="okx-secret"
COINBASE_API_KEY="cb-key"
COINBASE_API_SECRET="cb-secret"

# Database connection
DB_HOST="localhost"
DB_NAME="quantdb"
DB_USER="quant"
DB_PASS="strongpassword"

# Telegram for alerts
TELEGRAM_BOT_TOKEN="123456:ABC-DEF..."
TELEGRAM_CHAT_ID="987654321"
```

Этот файл хранится отдельно и не попадает в репозиторий. Загружается при старте контейнера/программы, после чего ключи доступны в конфигурации.

Docker-compose.yml (структура):

```
version: "3.8"
services:
  db:
    image: timescale/timescaledb:latest-pg15
    environment:
```

```

    POSTGRES_PASSWORD: ${DB_PASS}
    POSTGRES_USER: ${DB_USER}
    POSTGRES_DB: ${DB_NAME}
volumes:
  - ./data/db:/var/lib/postgresql/data
restart: unless-stopped

app:
  build: .
  env_file: .env
  depends_on:
    - db
  links:
    - db
  volumes:
    - ./app:/app
  command: ["bash", "-c", "python run_bot.py"]
  restart: unless-stopped

grafana:
  image: grafana/grafana:latest
  volumes:
    - ./grafana:/var/lib/grafana
  ports:
    - "3000:3000"

prometheus:
  image: prom/prometheus:latest
  volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml
  ports:
    - "9090:9090"

```

Здесь определены три основных контейнера: база Timescale, приложение бота, мониторинг (Grafana+Prom). App зависит от DB. Развернув compose, система готова.

Псевдокод: инкрементальная синхронизация OHLCV:

```

def sync_ohlc(ccxt_client, symbol, timeframe):
    # Получаем время последней свечи в БД
    last_ts = db.fetch_last_timestamp(symbol, timeframe)
    # Запрашиваем новые свечи с последней точки (exclusive)
    new_candles = ccxt_client.fetch_ohlc(symbol, timeframe, since=last_ts +
1)
    # Записываем в БД, избегая дубликатов
    for ohlc in new_candles:
        db.upsert_candle(symbol, timeframe, ohlc)
    if new_candles:
        print(f"{len(new_candles)} new candles synced for {symbol}-{
{timeframe}}")

```

Этот метод вызывается по расписанию (например, каждую минуту) для каждого символа. Он берет последнюю запись, догружает с биржи всё новое и складывает. За счёт `since=last_ts`, уже имеющиеся не придут. `Duplicates upsert` не создаст дублей.

Псевдокод: pipeline сигналов:

```
def compute_indicators(data):
    # data: DataFrame с колонками ['open', 'high', 'low', 'close', 'volume']
    features = {}
    features['fast_ma'] = talib.SMA(data['close'], timeperiod=50)
    features['slow_ma'] = talib.SMA(data['close'], timeperiod=200)
    features['rsi'] = talib.RSI(data['close'], timeperiod=14)
    return features

def generate_signal(features):
    # Пример стратегии: MA crossover + RSI filter
    if features['fast_ma'][-1] > features['slow_ma'][-1] and features['rsi'][-1] < 70:
        return 1 # buy
    elif features['fast_ma'][-1] < features['slow_ma'][-1] and features['rsi'][-1] > 30:
        return -1 # sell
    else:
        return 0

# Использование
data = db.load_recent_data("BTC/USDT", limit=300)
feat = compute_indicators(data)
signal = generate_signal(feat)
if signal != 0:
    place_trade("BTC/USDT", signal)
```

Здесь `place_trade` – функция, которая через OMS разместит ордер нужного типа (например, маркет на весь доступный объем по рассчитанному размеру).

Псевдокод: отправка ордера и reconcile:

```
open_orders = {} # хранит исходящие активные ордера

def submit_order(symbol, side, qty, price=None, type="MARKET"):
    client_order_id = str(uuid4())
    try:
        resp = exchange.create_order(symbol, type, side, qty, price,
{'clientOrderId': client_order_id})
    except Exception as e:
        log.error(f"Order send failed: {e}")
        # Если ошибка сети или unknown, планируем повтор через 1с
        schedule_task(1, reconcile_order_status, client_order_id, symbol)
    return
```

```

    open_orders[client_order_id] = {'symbol': symbol, 'side': side, 'qty':
qty}
    log.info(f"Submitted {side} {qty} {symbol}, client_id={client_order_id}")

def reconcile_order_status(client_id, symbol):
    # Проверяем статус по бирже
    order = exchange.fetch_order_by_client_id(client_id, symbol)
    if order is None:
        log.warning(f"No info for order {client_id} - assume not placed")
        return
    if order['status'] in ('closed', 'filled', 'canceled'):
        open_orders.pop(client_id, None)
        log.info(f"Order {client_id} completed with status
{order['status']}, filled {order['filled']}")
        if order['status'] == 'filled':
            update_position(symbol, order['filled'], order['side'])
    else:
        # Все еще открытый - проверим позже
        schedule_task(5, reconcile_order_status, client_id, symbol)

```

Объяснение: При отправке ордера генерируется уникальный ID, отдаётся бирже. Если вызов вернул ошибку, запускаем reconcile через 1 секунду – возможно, ордер таки выставился. В `reconcile_order_status` пытаемся получить ордер по client_id (не все биржи это умеют; если нет – можно по символу искать). Если статус конечный – убираем из трекинга и обновляем локальное состояние (позицию). Если всё ещё открытый – планируем повторную проверку через 5 сек.

Список поставщиков данных и ПО (Vendors & Open Source):

• Данные (Data Vendors):

- *Kaiko*: институт. провайдер – полный исторический ордербук и трейды, высокое качество, высокая цена (подходит если нужен бэкфилл тиков за годы) ⁸².
- *CoinAPI*: унифицированный API ~350 бирж; платно, но удобно. Предоставляет OHLCV, трейды, ордербуки. Можно купить пакет исторических данных one-off ⁸³.
- *CryptoCompare*: API для историч. свечей (много бесплатного с ограничениями), а также потоки через вебсокет. Хорош для бэкап-источника (например, если пропущены данные, можно дернуть CryptoCompare).
- *CoinGecko*: бесплатный публичный API, но **только справочные данные** (цена last, объёмы, tickers). Используется для получения списка монет, метаданных (логотип, категория), а также курсов фиат <-> крипто. Ограничен rate limit (но у нас редкие вызовы).
- *Binance Public Data*: Binance выложил в S3 всю историю торгов и свечей. Можно скачивать по HTTP бесплатно (но объёмы большие). Можно заскриптовать для начального наполнения БД.
- *Poligon.io, AlphaVantage*: не для крипто (скорее акции/forex), но у AlphaVantage есть и крипто API (бесплатно, но с низкой частотой). Можно использовать как резерв для основных пар.

• Библиотеки и фреймворки (Libraries):

- *CCXT*: версия ~3, основной интерфейс к биржам. Поддерживает REST (торговля, данные), частично WebSocket (в платной версии CCXT Pro). Мы возьмём CCXT для REST и, возможно, ccxt.async поддержка. Альтернатива – *XChange* (Java) или напрямую API – нецелесообразно. CCXT даёт единый словарь полей и ручки ошибок.

- *Pandas, NumPy*: фундамент для работы с табличными данными, вычислений. Pandas 2.0 (с Arrow) ещё ускорит работу. Numpy 1.25+. Эти используются во всех частях (от индикаторов до формирования матриц фич).
- *TA-Lib / pandas-ta*: TA-Lib – проверенная библиотечка на C, но нужно устанавливать биндинги. `pandas_ta` – pure Python, но внутри часто использует vectorized ops. Мы можем использовать `pandas_ta` за удобство (pip install) – он содержит TA-Lib индикаторы + свои.
- *VectorBT*: версия 0.24 (2023) – будем применять для research (не runtime). Пишем исследования, генерируем сигналы/статы. В прод-код бот может не тянуть её (чтобы не перегружать), но для бэктеста-модуля – да.
- *Scikit-Learn*: 1.2 – для простых ML моделей (логрег, SVM) если понадобятся. Легко интегрировать для offline тренировки. В runtime можно тоже inference с sklearn, если модель лёгкая. Для нейросетей – *PyTorch 2.x* (TensorFlow возможно overkill) – PyTorch хорош, если будем делать LSTM или CNN для предсказаний.
- *MLflow / DVC*: для управления экспериментами и данными. MLflow 2.x – можем использовать для трекинга параметров модели, но т.к. проект небольшой, возможно достаточно вручную. DVC – version control для данных, можно применять для дата-сетов (например, хранить бэкстест результаты). Пока возможно overkill.
- *FastAPI*: 0.95 – может быть использован, если делаем API или простенький веб-интерфейс. Например, endpoint `/status` дающий JSON состояния (для мониторинга). Или serve модели (но у нас бот – не сервис для клиентов, web UI не обязателен).
- *Starlette / Uvicorn*: сопутствуют FastAPI, если пойдём тем путём.
- *Prometheus client*: library (prom-client) для Python, чтобы экспортировать метрики (или use PushGateway).
- *Python-telegram-bot*: библиотека для отправки сообщений в Telegram (для алертов).
- **Инфраструктура (Infra/DevOps)**:
 - *Docker*: используем Docker для контейнеризации – от БД до приложения. Версия 24.x. Docker Compose – для упрощённого оркестрирования.
 - *Kubernetes*: пока не нужен. Но если понадобится масштаб – платформа должна быть контейнеризирована, так что затем можно на k8s (AKS/EKS) развернуть.
 - *Git & CI*: репозиторий Git (например, на GitLab или GitHub private). CI – GitHub Actions или GitLab CI.
 - *NGINX*: Если будет веб-дэшборд (Grafana) и нужно TLS, ставим Nginx Proxy. Но может не потребоваться, если прямой доступ или via SSH туннель.
 - *PostgreSQL / Timescale*: TimescaleDB 2.11 (на PG15).
 - *Redis*: может быть использован для: кэш цен, брокер Celery (если Airflow) или Pub/Sub между сервисами. В текущем дизайне monolith – необязательно. Но можно задействовать для rate-limiting (incr key on each API call).
 - *Airflow/Prefect*: выбираем Prefect 2, Cloud.
 - *Grafana, Prometheus*: Grafana v10, Prom v2.
 - *Sentry*: опционально для сбора ошибок runtime (но т.к. мы сами мониторим, можно и без).
- **Execution Libraries**:
 - *CCXT (execution)*: уже упомянули.
 - *WebSocket libs*: `websockets` or `aiohttp` for streaming. Some exchanges have official websockets Python (e.g., Binance has python-binance).
 - *Hummingbot connectors*: Опционально, Hummingbot открыты коннекторы, можно позаимствовать. Но они integrated with Hummingbot, лучше CCXT + отдельные wss.
 - *AsyncIO*: Python async for concurrency (feed handling).
 - *PyHibernator/HFT libs*: not needed since no HFT, but note: there exist C++ libs (like QuickFAST for OPRA) – out of scope.

В целом, ставка на **open-source**: все компоненты бесплатны, с активным комьюнити. Это позволит поддерживать платформу актуальной и гибко менять.

11. Предлагаемый стек технологий (Proposed Tech Stack)

Ниже приведён выбор основных компонентов по слоям с указанием версий и обоснованием, а также запасные варианты:

- **Язык программирования:** *Python 3.11* – оптимальный для data science и быстрого прототипирования. Поддерживает asyncio для concurrency, большая экосистема библиотек для ML/статистики. *Альтернатива:* C# (QuantConnect Lean) – отклонено из-за худшей гибкости для ресерча; JavaScript (Node) – не подходит для чисел; C++ – слишком долго разрабатывать и трудно менять стратегии на лету.
- **Сбор и подключение к API:** *CCXT 3.x* – унифицированная библиотека для бирж (REST). Обеспечит быстрое подключение Binance, OKX, Coinbase ⁵⁵. Версия 3 или актуальная. *Fallback:* использовать binance-api-python, okx-python-SDK по отдельности (нежелательно, потеря унификации).
- **Streaming/WebSockets:** *Python websockets 11* для прямой работы с WS бирж (если нужно), либо *CCXT Pro* (но это платно). *Fallback:* использовать *SignalR for Coinbase* (specific). Основной – websockets, т.к. даёт низкоуровневый контроль.
- **База данных:** *TimescaleDB 2.11 (на PostgreSQL 15)* – избран для основного хранилища таймсерий и позиций. Причины: SQL-функционал, транзакционность, простота администрирования, хорошая производительность на временных рядах ¹⁶. *Fallback:* *PostgreSQL без Timescale* (если объёмы невелики, можно и так), или *ClickHouse 22* – если понадобится ускорить аналитические запросы (но тогда придется мириться с eventual consistency для онлайн).
- **Хранилище моделей/конфигураций:** *Git + DVC* (DVC optional). Модели (если будут, например .pkl) можно хранить как артефакты, версии в Git с LFS. *Fallback:* MLflow server (но overhead).
- **Фреймворк бэкестинга:** *VectorBT 0.24* – для исследований и тестов. Он будет использоваться в офлайн-скриптах, а не внутри живого бота (во время торговли не нужен). *Fallback:* *Backtrader* для некоторых live-сценариев (но, скорее всего, реализуем живую торговлю без него).
- **Библиотеки анализа и ML:** *pandas 2.0, NumPy 1.25, pandas-ta 0.3, TA-Lib 0.4* (если нужно более быструю на C). *SciPy 1.11* (для статист. функций). *Scikit-learn 1.2* и *PyTorch 2.0* – для ML моделей (в будущем).
- **Оркестрация и планировщик:** *Prefect 2.10* (или актуал.) – выбрано за простоту и гибкость. Plan: initial flows run in Prefect Cloud (free), при желании можно перейти на локальный Prefect Orion. *Fallback:* *Apache Airflow 2.6* – если понадобится более сложный DAG и есть ресурс поддерживать, но для одного пользователя Prefect достаточно.
- **Развертывание:** *Docker* – контейнеризация приложения и Timescale. *Docker Compose* – для локального и серверного запуска. Версии latest стабильные. *Fallback:* прямой запуск на VM (но тогда сложнее переносимость и обновление).
- **CI/CD:** *GitHub Actions* – для тестов, линтеров, деплоя. *Fallback:* GitLab CI (если репо на GitLab).
- **Мониторинг:** *Prometheus 2.x* (с NodeExporter, если хотим метрики системы), *Grafana 10*. *Alertmanager* для уведомлений. Стек настраиваемый. *Fallback:* *Datadog* (платно) или *Zabbix* (но Prometheus+Grafana сейчас индустриальный standard OSS).
- **Логирование:** *Стандартный логгер Python* + ротация (logging.handlers). Формат JSON (для возможно парсинга). *Fallback:* *ELK stack* – но это overhead; вместо, можно Grafana Loki как простой вариант (пока не требуется).

- **Алерты:** *Telegram Bot API* – для отправки сообщений о важных событиях. Уже есть lib `python-telegram-bot`.
- **Безопасность:** *Fail2ban* на сервере (SSH guard), *ufw/iptables* – firewall, *OpenVPN* или *Tailscale* для доступа (devOps side). Secrets management: `.env` + желательно *docker secrets* (но в одном узле `.env` норм).
- **API/Interface (опционально):** Если нужен REST API для платформы – используем *FastAPI 0.95*. Но, вероятно, не нужно выдавать наружу API. UI – Grafana dashboards достаточно. Возможно, простой CLI/телеграм команды (старт/стоп, статус).

Резервные опции и обоснования: Выбранные технологии – Python-centric, что ускорит разработку. Если бы скорость была критична, могли бы рассмотреть C++/Rust для некоторых модулей, но отказ: усложнение поддержки. Timescale vs ClickHouse: выбрали TimescaleDB, т.к. баланс удобства и скорости (и знакома SQL) – если увидим, что данных >100 млн и PG не тянет, можно добавить ClickHouse для агрегатов (например, исторические сводки). Prefect против Airflow: Prefect – очевидно лучше для 1 разработчика (Airflow overkill, мы не AirBnB).

Итого, стек представляет собой **полностью open-source решение**, охватывающее все уровни: от данных (Timescale) до логики (Python libs) и инфраструктуры (Docker, Prometheus). Он рассчитан на **надежность и воспроизводимость**: вся среда описана в коде (compose, requirements). В случае сбоя или миграции – можно быстро поднять на другом сервере. За каждым компонентом стоят крупные сообщества (это снижает риск – можно получить поддержку/обновления).

Этот тщательно подобранный стек позволит единственному инженеру построить платформу уровня хедж-фонда, сохранив при этом разумные затраты и контроль над системой. Все решения сопровождаются источниками или обоснованиями, подтверждающими их актуальность на Q4 2025.

Источники (SOURCES):

1. Vishal Menon, *"Building Next-Generation Data Infrastructure for Crypto Analytics,"* Medium, Jun. 19, 2025 [1](#) [2](#) [6](#) .
2. TokenMetrics Team, *"Choosing the Right Crypto API for Your Bot: REST vs WebSockets Explained,"* TokenMetrics Blog, Oct. 2025 [8](#) [3](#) .
3. GitHub – johnbdfilio000, *Crypto Data Ingestion Module (README)*, 2023 [9](#) [10](#) [84](#) .
4. TigerData, *"Not All Analytics Are Equal: Benchmarking Databases for Real-Time Analytics,"* Oct. 2025 [16](#) .
5. Coinpaprika (BH), *"API Keys in Crypto: Invisible Passwords Powering Exchanges,"* Sep. 19, 2025 [51](#) [52](#) .
6. Investopedia, *"4 Big Risks of Algorithmic High-Frequency Trading,"* Jan. 27, 2016 [5](#) .
7. CoinAPI Blog, *"13 Advanced Order Types That Can Increase Your Profits in Crypto,"* Aug. 2023 [40](#) [41](#) .
8. Michael Preston, *"Airflow vs Prefect vs Dagster – no-nonsense comparison,"* Medium, Oct. 2025 [73](#) [48](#) [74](#) .
9. Trading Dude, *"Battle-Tested Backtesters: VectorBT vs Zipline vs Backtrader,"* Medium, Sep. 2025 [35](#) [61](#) .
10. Scribd (HarmoniQ Insights), *"Quantitative Alpha in Crypto Markets"* (research review), 2025 [20](#) [19](#) .
11. José C. G. Tanaka, *"A novel drift detection algorithm for ML in trading (ADDMM),"* QuantInsti Blog, Mar. 23, 2025 [49](#) .
12. Austin Werner, *"Crypto Risk Management: Essential Strategies for 2025,"* Blog, Mar. 13, 2025 [85](#) [37](#) .
13. Reddit r/Database, *"Best DB for high-ingestion time-series?"*, Jul. 2025 [67](#) [86](#) .

14. IOSCO Tech. Report, *"Issues Raised by Technological Changes on Market Integrity,"* Jul. 2011 ⁸⁷ (via Investopedia).
 15. FreeAgent Engineering, *"Decoding Orchestration: Prefect vs Dagster vs Airflow,"* Jul. 2025 ⁸⁰ ⁸¹ .
-

1 2 6 Building Next-Generation Data Infrastructure for Crypto Analytics | by Vishal Menon | Medium

<https://vishal10menon.medium.com/building-next-generation-data-infrastructure-for-crypto-analytics-b62d071fa225>

3 7 8 REST vs WebSocket Crypto: API Comparison for Bots in 2025

https://www.tokenmetrics.com/blog/crypto-api-bot-rest-vs-websockets?0fad35da_page=72&74e29fd5_page=19

4 Daily Loss Limits Explained: Protect Your Account and Sanity

<https://myfundedfutures.com/blog/daily-loss-limits-explained-protect-your-account-and-sanity>

5 87 4 Big Risks of Algorithmic High-Frequency Trading

<https://www.investopedia.com/articles/markets/012716/four-big-risks-algorithmic-highfrequency-trading.asp>

9 10 15 84 GitHub - johnbdfilio000/crypto-data-ingestion-module: A comprehensive Python-based crypto market data collector that fetches historical and live OHLCV (Open, High, Low, Close, Volume) candlestick data from multiple cryptocurrency exchanges. It supports various intervals and symbols, storing data efficiently in Parquet format for easy analysis

<https://github.com/johnbdfilio000/crypto-data-ingestion-module>

11 12 13 14 65 66 69 70 ClickHouse vs TimescaleDB. What is the difference? A detailed comparison | by Data Engineer | DoubleCloud | Medium

<https://medium.com/doublecloud-insights/clickhouse-vs-timescaledb-what-is-the-difference-a-detailed-comparison-62127a989d8d>

16 Not All Analytics Are Equal: Benchmarking Databases for Real-Time Analytics Applications | TigerData

<https://www.tigerdata.com/blog/benchmarking-databases-for-real-time-analytics-applications>

17 67 68 86 Best database for high-ingestion time-series data with relational structure? : r/Database

https://www.reddit.com/r/Database/comments/1labnhv/best_database_for_highingestion_timeseries_data/

18 30 31 Features - vectorbt

<https://vectorbt.dev/getting-started/features/>

19 20 63 SSRN 5225612 | PDF | Machine Learning | Cryptocurrency

<https://www.scribd.com/document/875675598/ssrn-5225612>

21 The Triple Barrier Method: A New Standard for Investment Labeling ...

<https://medium.datadriveninvestor.com/the-triple-barrier-method-a-new-standard-for-investment-labeling-and-analysis-1a525a0a2f46>

22 The Hidden Trap in Algorithmic Trading: Data Leakage in Backtesting | by Unicorn Day | Medium

<https://medium.com/@wl8380/the-hidden-trap-in-algorithmic-trading-data-leakage-in-backtesting-622a13e01cb9>

23 24 Cross Validation in Finance: Purging, Embargoing, Combinatorial

<https://blog.quantinsti.com/cross-validation-embargo-purging-combinatorial/>

25 Meta Labeling for Algorithmic Trading: How to Amplify a Real Edge

https://www.reddit.com/r/algotrading/comments/1lnm48w/meta_labeling_for_algorithmic_trading_how_to/

26 27 28 29 35 36 60 61 62 Battle-Tested Backtesters: Comparing VectorBT, Zipline, and Backtrader for Financial Strategy Development | by Trading Dude | Medium

<https://medium.com/@trading.dude/battle-tested-backtesters-comparing-vectorbt-zipline-and-backtrader-for-financial-strategy-dee33d33a9e0>

32 Handling Data - Crypto Futures - QuantConnect.com

<https://www.quantconnect.com/docs/v2/writing-algorithms/securities/asset-classes/crypto-futures/handling-data>

33 Requesting Data - Crypto - QuantConnect.com

<https://www.quantconnect.com/docs/v2/writing-algorithms/securities/asset-classes/crypto/requesting-data>

- 34 The Combinatorial Purged Cross-Validation method - Towards AI
<https://towardsai.net/p/l/the-combinatorial-purged-cross-validation-method>
- 37 85 Crypto Risk Management Essential Strategies | Austin Werner
<https://austinwerner.io/blog/risk-management-crypto>
- 38 Results after 100+ crypto trades using a Trend-following strategy ...
https://www.reddit.com/r/algotrading/comments/ww639t/results_after_100_crypto_trades_using_a/
- 39 10 Risk Management Strategies for Crypto Trading Success
<https://bitlearn.network/10-risk-management-strategies-for-crypto-trading-success/>
- 40 41 42 44 45 CoinAPI.io Blog - 13 Advanced Order Types That Can Increase Your Profits in Crypto
<https://www.coinapi.io/blog/13-advanced-order-types>
- 43 Understanding the order types | Coinbase Help
<https://help.coinbase.com/en/coinbase/trading-and-funding/advanced-trade/order-types>
- 46 47 48 72 73 74 75 76 77 78 79 80 81 Airflow vs Dagster vs Prefect: Data Orchestrator Comparison | Galaxy
<https://www.getgalaxy.io/learn/glossary/airflow-vs-dagster-vs-prefect-choosing-the-right-data-orchestrator>
- 49 50 Autoregressive Drift Detection Method (ADDM) in Trading: Concept Drift, Market Regimes, and Strategy
<https://blog.quantinsti.com/autoregressive-drift-detection-method/>
- 51 52 53 54 API Keys in Crypto: The Invisible Passwords Powering Exchanges and Apps
<https://coinprika.com/education/api-key-crypto-definition-uses-security/>
- 55 freqtrade/freqtrade: Free, open source crypto trading bot - GitHub
<https://github.com/freqtrade/freqtrade>
- 56 Hummingbot - the open source framework for crypto market makers ...
<https://hummingbot.org/>
- 57 58 59 NautilusTrader
<https://nautilustrader.io/>
- 64 71 How TimescaleDB helped us scale analytics and reporting
<https://blog.cloudflare.com/timescaledb-art/>
- 82 [PDF] benchmarking specialized databases for high-frequency data - arXiv
<https://arxiv.org/pdf/2301.12561>
- 83 Handling Historical Crypto Data Volumes: How CoinAPI's S3 and ...
<https://www.coinapi.io/blog/handling-historical-crypto-data-volumes-how-coinapi-s-s3-and-flat-files-simplify-scale>